HOSPITAL MANAGEMENT SYSTEM

By

Ali Hamza

REGISTRATION NO.: 2021-GCUF-066949

This project has been submitted as a
partial requirement for the
completion of the

BACHELOR OF SCIENCE IN
COMPUTER SCIENCE

DEPARTMENT OF COMPUTER SCIENCE

GC UNIVERSITY, FAISLABAD

AUGUST 2025

ACKNOWLEDGEMENTS

First of all, we respectfully thank Allah Almighty, the Merciful and the Beneficent, who gave us health, thoughts, wisdom and cooperative people to enable us achieve this goal. We would like to express my sincere gratitude to our supervisor Prof. Arslan. Because he is not only the supervisor of our project he is also one of the best teachers we know. Prof. Arslan for providing the opportunity to work on this interesting project work. His constructive criticism during our project helped us to make necessary improvements. We also wanted to thanks our parents who inspired, encouraged and fully supported us for every trials that comes our way, we are grateful for their never ending moral support and prayers which always acted as a catalyst in our academic life.

Dedication

We dedicate this project to ALLAH ALMIGHTY our creator, our strong pillar, our source of inspiration, wisdom, knowledge and understanding. He has been the source of our strength throughout this program and through every aspect of our life.

We also dedicate it to our honorable supervisor Professor Arslan, also to the people behind this project whose efforts made our ideas to turn in to reality. It's a pray to ALLAH that give success and prosperity to all the peoples who helped us in the project.

We also dedicate to the source of our success and achievements to our families who raised us to be the person who we are today. Thank you for your unconditional love and support, you provided us with. Thank you for everything.

Name: Iqra Rani

Registration Number: 2021-GCUF-066927

Signature: _____

Name: Attiqa Kanwal

Registration No: 2021-GCUF-066942

Signature: _____

Name: Ali Hamza

Registration No: 2021-GCUF-066949

Signature: _____

SUPERVISOR SIGNATURES: _____

Table of Contents

CHAPTER 1 ..... **7**

INTRODUCTION TO THE PROBLEM **7**

1.1 INTRODUCTION 7

1.2 BACKGROUND 7

1.3 PURPOSE 8

1.4 SCOPE 8

1.5 OBJECTIVE 8

1.6 INTENDED AUDIENCE AND READING SUGGESTIONS 9

1.7 DOCUMENT CONVENTIONS 9

CHAPTER 2 ..... **10**

SOFTWARE REQUIREMENT SPECIFICATION **10**

2.1 OVERALL DESCRIPTION: 10

2.1.1 PRODUCT PERSPECTIVE 10

2.1.2 ASSUMPTIONS AND DEPENDENCIES ..... **Error! Bookmark not defined.**

2.1.3 DESIGN CONSTRAINTS 11

2.2 Class and characteristics 11

2.2.1. FUNCTIONAL REQUIREMENTS: 12

2.3 EXTERNAL INTERFACE REQUIREMENTS 13

2.3.1 HARDWARE INTERFACE 13

2.3.2 SOFTWARE INTERFACE 13

2.4 OTHER NONFUNCTIONAL REQUIREMENTS 13

2.4.1 PERFORMANCE REQUIREMENTS 13

2.4.2 SAFETY REQUIREMENTS 13

2.4.3 SECURITY REQUIREMENTS 13

CHAPTER 3 14

3.1 Identifying Use-cases using Textual Context 14

3.2 FORMINING USE CASE DIAGRAM WITH USECASES 16

3.3 DESCRIBE THE EVENTS FLOW FOR USE CASE 19

CHAPTER 4 ..... **24**

4.1 ARCHITECTURE DIAGRAM 24

4.2 ERD WITH DATA DICTIONARY 26

4.3 DATA FLOW DIAGRAM (LEVEL 0 AND LEVEL 1) 27

4.3.1 0 LEVEL DFD: 27

4.3.2 1 LEVEL DFD: 28

4.4 CLASS DIAGRAM 29
4.5 OBJECT DIAGRAM 30
4.6 SEQUENCE DIAGRAM 31
4.7 ACTIVITY DIAGRAM 32
4.8 COLLABORATION DIAGRAM 33
4.9 STATE TRANSITION DIAGRAM 34
5.1 COMPONENT DIAGRAM 35
5.2 DEPLOYMENT DIAGRAM 36
CHAPTER 6 38
TESTING (SOFTWARE QUALITY ATTRIBUTES) 38
6.1 TEST CASE SPECIFICATION 38
6.1.1 FUNCTIONAL TESTING 39
6.1.2 UNIT TESTING 40
6.1.3 INTEGRATION TESTING 41
6.1.4 SYSTEM TESTING 43
Testing Areas: 43
Non-Functional Testing: 45
Expected Outcomes: 45
6.1.5 REGRESSION TESTING 46
6.1.6 ACCEPTANCE TESTING 48
Test Scenarios: 49
User Experience Validation: 50
Non-Functional Testing: 50
Sign-Off Criteria: 50
6.1.7 ALPHA TESTING 51
6.1.8 BETA TESTING 51
6.1.9 NON-FUNCTIONAL TESTING 52
6.2 BLACK BOX TEST CASES 60
6.2.1 BVA OR BOUNDARY VALUES ANALYSIS 61
6.2.2 EQUIVALENCE CLASS PARTITIONING 62
6.2.3 STATE TRANSITION TESTING 62
6.2.4 DECISION TABLE TESTING 63
6.3 WHITE BOX TESTING 63
6.3.1 STATEMENT COVERAGE 64
6.3.2 BRANCH COVERAGE 65
6.3.3 PATH COVERAGE 65

TOOLS AND TECHNOLOGIES **66**
7.1 PROGRAMMING LANGUAGE 66
7.2.1. Components of Visual Studio Code for HMS Development 68
7.2.2. Operating Environment for HMS Development 69
APPENDIX A **70**
USER DOCUMENTATION **70**
APPENDIX B: **75**
SOURCE CODE **75**

LIST OF FIGURES

Figure: 3.2. 1 USE CASE DIAGRAM **12**
Figure: 4.1. 1 ARCHITECTURE DIAGRAM **21**
Figure: 4.1. 2 ERD WITH DATA DICTIONARY **22**
Figure: 4.1. 3 DATA FLOW DIAGRAM 0 LEVEL **23**
Figure: 4.1. 4 DATA FLOW DIAGRAM 1 LEVEL **24**
Figure: 4.1. 5 CLASS DIAGRAM **25**
Figure: 4.1. 6 OBJECT DIAGRAM **26**
Figure: 4.1. 7 SEQUENCE DIAGRAM **27**
Figure: 4.1. 8 ACTIVITY DIAGRAM **28**
Figure: 4.1. 9 COLLABORATION DIAGRAM **29**
Figure: 4.1. 10 STATE TRANSITION DIAGRAM **30**
Figure: 5.1. 1 Component Diagram **32**
Figure: 5.2. 1 Deployment Diagram **37**
Figure: 6.2. 1 BLACK BOX TEST CASE **60**
Figure: 6.3.1 WHITE BOX TESTING **62**

LIST OF TABLES

Table: 3.1. 1 IDENTIFY ACTORS AND USE CASE 11
Table: 3.3. 1 REGISTRARTION 16
Table: 3.3. 2 LOGIN 16
Table: 3.3. 3 BOOK APPOINTMENT 17
Table: 3.3. 4 VIEW APPOINTMENT HISTORY 17
Table: 3.3. 5 VIEW PRESCRIPTION 18
Table: 3.3. 6 MANAGE DOCTORS & USERS 18
Table: 3.3. 7 VIEW APPOINTMENTS(DOCTOR/ADMIN) 18
Table: 3.3. 8 DIAGNOSE PATIENT 19
Table: 3.3. 9 LOGOUT 19
Table: 6.1. 1 REGISTRATION TEST CASE 56
Table: 6.1. 2 ADD PATIENT CASE 57
Table: 6.1. 3 ADD DOCTOR TEST CASE 57
Table: 6.1. 4 DELETE DOCTOR TEST CASE 58
Table: 6.1. 5 APPOINTMENT VIEW TEST CASE 58
Table: 6.1. 6 MANAGE USERS TEST CASE 59
Table: 6.1. 8 LOGOUT 59

CHAPTER 1

INTRODUCTION TO THE PROBLEM

1.1 INTRODUCTION

Hospitals today face significant challenges in managing their operations efficiently. The complexity of healthcare services, coupled with the need for accurate record-keeping, timely decision-making, and effective resource allocation, demands a robust system. Traditional manual methods and disparate systems can lead to:

1. Inefficient patient registration and management
2. Inaccurate or lost medical records
3. Poor staff coordination and communication
4. Ineffective inventory management and resource utilization
5. Delays in billing and insurance claims

1.2 BACKGROUND

Hospitals face numerous challenges in managing operations efficiently. Manual processes, paper-based records, and fragmented systems lead to inefficiencies, errors, and delays. A Hospital Management System (HMS) is essential to streamline operations, improve patient care, and reduce costs. The benefits of HMS include improved patient care, increased efficiency, reduced costs, and data driven decision making. By implementing an HMS, hospitals can significantly enhance their overall performance and quality of care.

Challenges:

1. Inefficient Patient Management
2. Disorganized Clinical Data
3. Poor Resource Allocation
4. Inaccurate Billing and Insurance Claims

1.3 PURPOSE

A Hospital Management System (HMS) is designed to manage and integrate various hospital operations, including patient care, administrative tasks, and financial management. The primary objective of an HMS is to improve the overall efficiency, effectiveness, and quality of hospital services. An HMS achieves this by automating administrative tasks, such as patient registration, billing, and inventory management, and optimizing resource allocation, including staff, equipment, and supplies. It also provides real-time data and analytics to support informed decision-making, improve quality of care, and optimize hospital operations. Overall, an HMS plays a critical role in modernizing hospital operations, improving patient care, and enhancing the overall quality of healthcare services.

1.4 SCOPE

A Hospital Management System (HMS) encompasses a broad range of functionalities designed to manage and integrate various hospital operations. This includes patient registration and management, electronic health records (EHRs), treatment planning, medication management, billing and insurance claims, staff management, inventory control, and equipment maintenance. The scope of an HMS extends to various departments within a hospital, including outpatient and inpatient services, emergency departments, laboratories, and pharmacies. By streamlining processes, reducing errors, and improving resource allocation, an HMS plays a critical role in modernizing hospital operations and enhancing the overall quality of healthcare services.

1.5 OBJECTIVE

A Hospital Management System (HMS) aims to leverage technology to improve the efficiency, quality, and safety of healthcare services. It provides accurate and timely access to patient information to support informed decision-making and high-quality care. By automating administrative tasks, optimizing resource allocation, and reducing waste, an HMS improves productivity and efficiency.

It implements safety protocols and alerts to minimize medical errors and adverse events, ensuring a safe and reliable care environment. An HMS also provides a seamless and personalized experience for patients through efficient registration, and scheduling.

1.6 INTENDED AUDIENCE AND READING SUGGESTIONS

A Hospital Management System (HMS) is designed for various stakeholders, including healthcare professionals, hospital administrators, patients, and hospital staff. Healthcare professionals use the system to access patient information, manage care, and coordinate treatment. Hospital administrators utilize the system to manage hospital operations, resources, and finances. Patients benefit from improved care, safety, and experience. Front-desk staff, billing personnel, and other support staff use the system for patient registration, billing, and administrative tasks.

1.7 DOCUMENT CONVENTIONS

Convention for Main Title

- Font face: Time New Roman
- Font Style: Bold
- Font Size :16

Convention for Sub Titles

- Font face: Time New Roman
- Font Style: Bold
- Font Size :14

Convention for Body

- Font face: Time New Roman
- Font Size: 12

CHAPTER 2

SOFTWARE REQUIREMENT SPECIFICATION

2.1 OVERALL DESCRIPTION:

2.1.1 PRODUCT PERSPECTIVE

The Hospital Management System (HMS) is a centralized, web-based application designed to automate and streamline hospital operations. It serves as a unified platform for administrators, doctors, and patients, ensuring seamless communication, accurate data management, and efficient service delivery.

The system is built using PHP for backend processing, MySQL for data storage, and Bootstrap/jQuery for a responsive and user-friendly interface. The HMS integrates multiple functional modules such as doctor management, patient records, appointment scheduling, medical history tracking, and messaging.

The system works in a role-based access environment:

1. Administrators manage doctors, patients, appointments, and general system configuration.
2. Doctors access patient records, update medical histories, and manage appointments.
3. Patients can register, log in, book appointments, and view their medical history.

This architecture eliminates the need for separate, disconnected tools, offering a single, secure, and accessible platform that improves efficiency, reduces paperwork, and enhances patient care.

2.1.2 ASSUMPTIONS AND DEPENDENCIES

a. The system will be deployed on a reliable web server with PHP and MySQL installed.
b. Users will have basic knowledge of operating a web browser.
c. Internet connectivity will be available for system access.
d. Users will provide accurate and valid credentials.
e. All system inputs will be validated to prevent errors.

The Dependencies are:

a. The system's backend database will be hosted on MySQL.
b. A functional PHP server environment (e.g., XAMPP, WAMP, or LAMP) is required.
c. Browser compatibility is essential — the system supports Chrome, Firefox, and Edge.
d. Bootstrap and jQuery dependencies must be included for frontend operation.

2.1.3 DESIGN CONSTRAINTS

The functional requirements are activities that the system must be able to perform. The system Provides he following functionalities:

1. The system shall allow registration of new patients and update patient demographics.
2. The system shall enable scheduling, rescheduling, and cancellation of patient appointments.
3. The system shall allow creation, storage, and retrieval of EHRs, including medical history, diagnoses, treatments, and test results.
4. The system shall enable generation of bills, tracking of payments, and management of insurance claims.
5. The system shall allow management of medical supplies, equipment, and pharmaceuticals.
6. The system shall provide reports and analytics on patient care, hospital operations, and financial performance.
7. The system shall allow management of user roles, access, and permissions.
8. The system shall ensure confidentiality, integrity, and availability of patient data and restrict access to authorized personnel.
9. The system shall send notifications and alerts for appointments, test results, and other important events.
10. The system shall integrate with other hospital systems, such as laboratory information systems or radiology information systems.

2.2 Class and characteristics

Class:

1. Patient: Represents a patient in the hospital, with attributes such as patient ID, name, date of birth, and medical history**.**

2. Staff: Represents a hospital staff member, with attributes such as staff ID, name, role, and contact information**.**

3. Appointment: Represents a scheduled appointment between a patient and a staff member, with attributes such as appointment date, time, and location.

4. Medical Record: Represents a patient's medical record, with attributes such as medical history, diagnoses, treatments, and test results.

5. Medication Represents a medication prescribed to a patient, with attributes such as medication name, dosage, and frequency**.**

2.2.1. FUNCTIONAL REQUIREMENTS:

i. The system should allow for registration, admission, discharge, and transfer of patients.
ii. The system should enable scheduling, rescheduling, and cancellation of patient appointments.
iii. The system should allow for creation, storage, and retrieval of electronic health records (EHRs).
iv. The system should enable generation of bills, tracking of payments, and management of insurance claims.
v. The system should allow for management of medical supplies, equipment, and pharmaceuticals.
vi. The system should provide reports and analytics on patient care, hospital operations, and financial performance.
vii. The system should allow for management of user roles, access, and permissions.
viii. The system should ensure confidentiality, integrity, and availability of patient data and comply with relevant regulations.
ix. The system should integrate with other hospital systems, such as laboratory information systems or radiology information systems.
x. The system should send notifications and alerts for appointments, test results, and other important events.

2.3 EXTERNAL INTERFACE REQUIREMENTS

2.3.1 HARDWARE INTERFACE

i. Any PC, Laptop Browser
ii. Mobile devices (In Future)

2.3.2 SOFTWARE INTERFACE

i. Operating System in PCs, Laptops etc.
ii. Language
iii. Xampp SQL Database

2.4 OTHER NONFUNCTIONAL REQUIREMENTS

2.4.1 PERFORMANCE REQUIREMENTS

i. The system shall respond to user queries within 2 seconds.
ii. The system shall process 1000 patient records per hour without data loss or degradation.
iii. The system shall be available 99.9% of the time, with minimal downtime for maintenance and update.

2.4.2 SAFETY REQUIREMENTS

i. The system shall ensure confidentiality and protection of patient data, in compliance with relevant regulations.
ii. The system shall implement role-based access controls to restrict access to authorized personnel.
iii. The system shall have a robust backup and recovery mechanism to prevent data loss in case of system failure.

2.4.3 SECURITY REQUIREMENTS

Verify user identity, limit access to authorized users, encrypt patient data, track system activity, monitor data access.

CHAPTER 3
ANALYSIS

3.1 Identifying Use-cases using Textual Context

A table in the context of use cases is a text based, structured format for documenting one or more use cases. Although there is no universal format for use case tables, organizations often have one or more templates to use as a starting point. A table can serve as a container for a series of high-level use cases or as a template for detailed individual use cases. Tables can supplement detailed UML use case diagrams or be used in their place. Regardless of the table's format, each use case will usually have a few key fields:

ID: Each use case should have a unique identifier.

Primary Actor: A use case should have one (and only one) primary actor. Description: Even the simplest use case table should have a sentence or two to describe each use case. Use case diagrams are a set of use cases, actors and their relationships

Use case

Admin:

i. Admins can log in to the system using their email and password to access the admin dashboard.
ii. Admins can edit their profiles and change their account passwords.
iii. Admins can manage users.
iv. Admins can manage billings of patients.
v. Admins can add, edit, or delete doctor profiles to manage available medical staff.
vi. Admins can view all scheduled appointments to monitor hospital activities.
vii. Admins can view the list of all registered patients to maintain medical records.
viii. Admins can update the landing page of the website, add blogs & upcoming announcements.

Doctor:

i. Doctors can log in to the system securely using their credentials to access their dashboard.
ii. Doctors can view their upcoming and past appointments to manage their schedule.
iii. Doctors can edit their profiles and change their account passwords.
iv. Doctors can add diagnoses to a patient's medical history after an appointment.
v. Doctors can prescribe medicines and instructions, which are saved to the patient's record.

Patient:

i. Patients can log in to the system using their credentials to access personal medical services.
ii. Patients can book appointments with available doctors by selecting dates, time, and specialization.
iii. Patients can edit their profiles and change their account passwords.
iv. Patients can view their past and upcoming prescriptions issued by doctors.
v. Patients can
vi. provide feedback regarding the hospital system.

Variables	**Description**
Admin	System administrator who manages users, doctors, and appointments
Doctor	Medical practitioner who attends appointments, diagnoses patients, and prescribes.
Patient	End user who books appointments and views prescriptions

Table: 3.1. 1 IDENTIFY ACTORS AND USE CASE

3.2 FORMINING USE CASE DIAGRAM WITH USECASES

A use case diagram is usually simple. It does not show the detail of the use cases:

1. It only summarizes some of the relationships between use cases, actors, and systems.
2. It does not show the order in which steps are performed to achieve the goals of each use case.
3. The main actors of **Hospital Management System** in this use case diagram.

The Relationship between actor and among the Use Cases.A standard form of use case diagram is defined in the Unified Modeling Language as shown in the Use Case Diagram below.

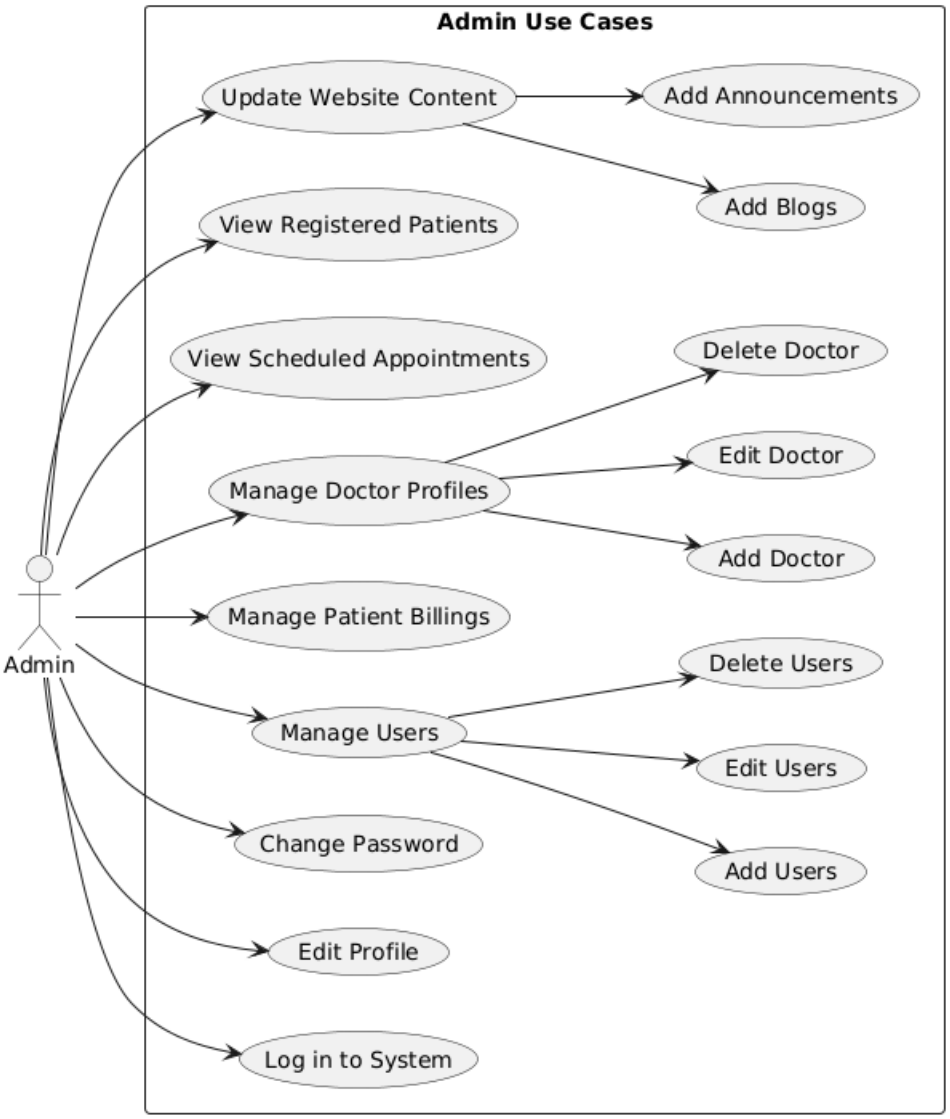


Figure: 3.2. 1 USE-CASE DIAGRAM

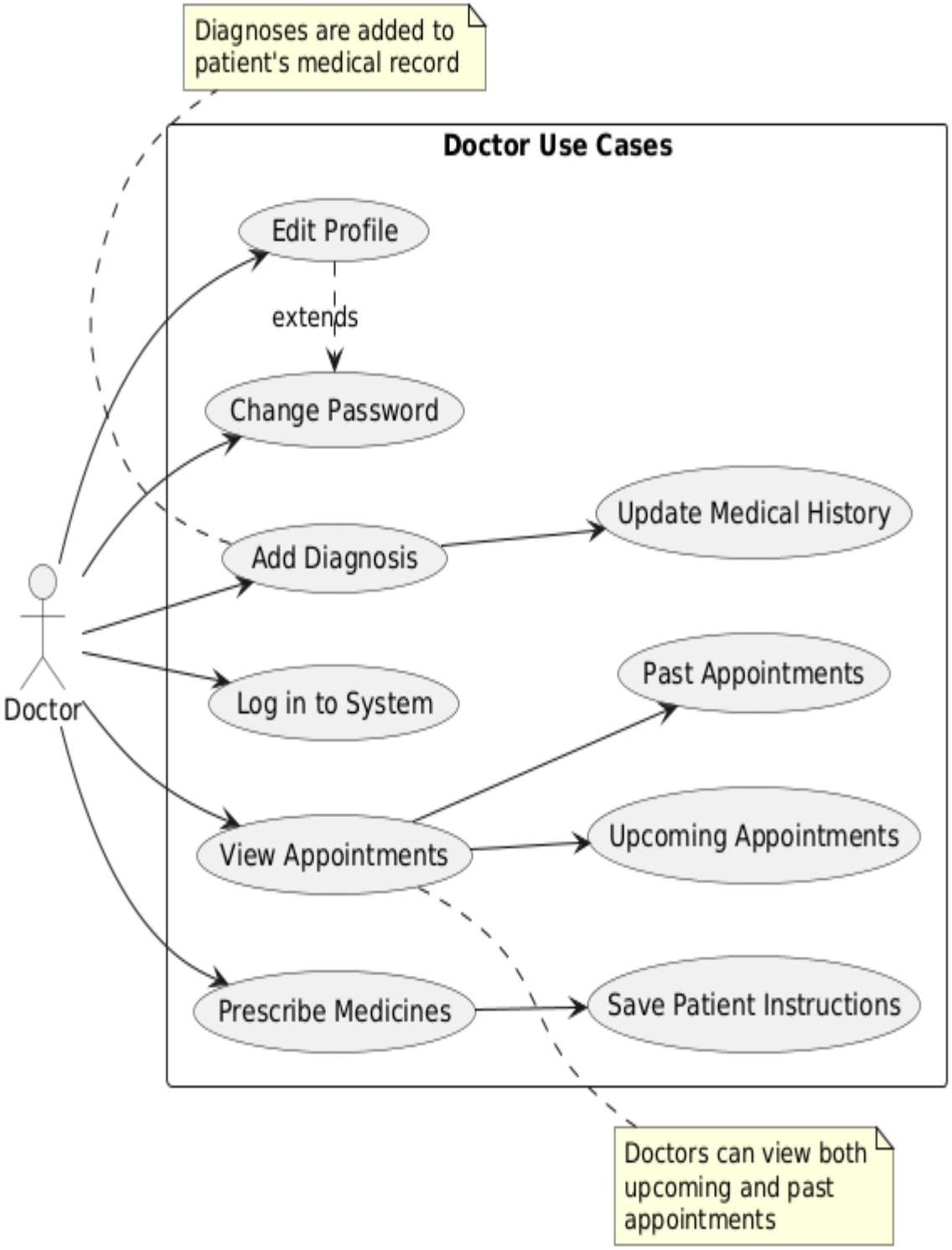


Figure: 3.2.2 USE CASE DIAGRAM

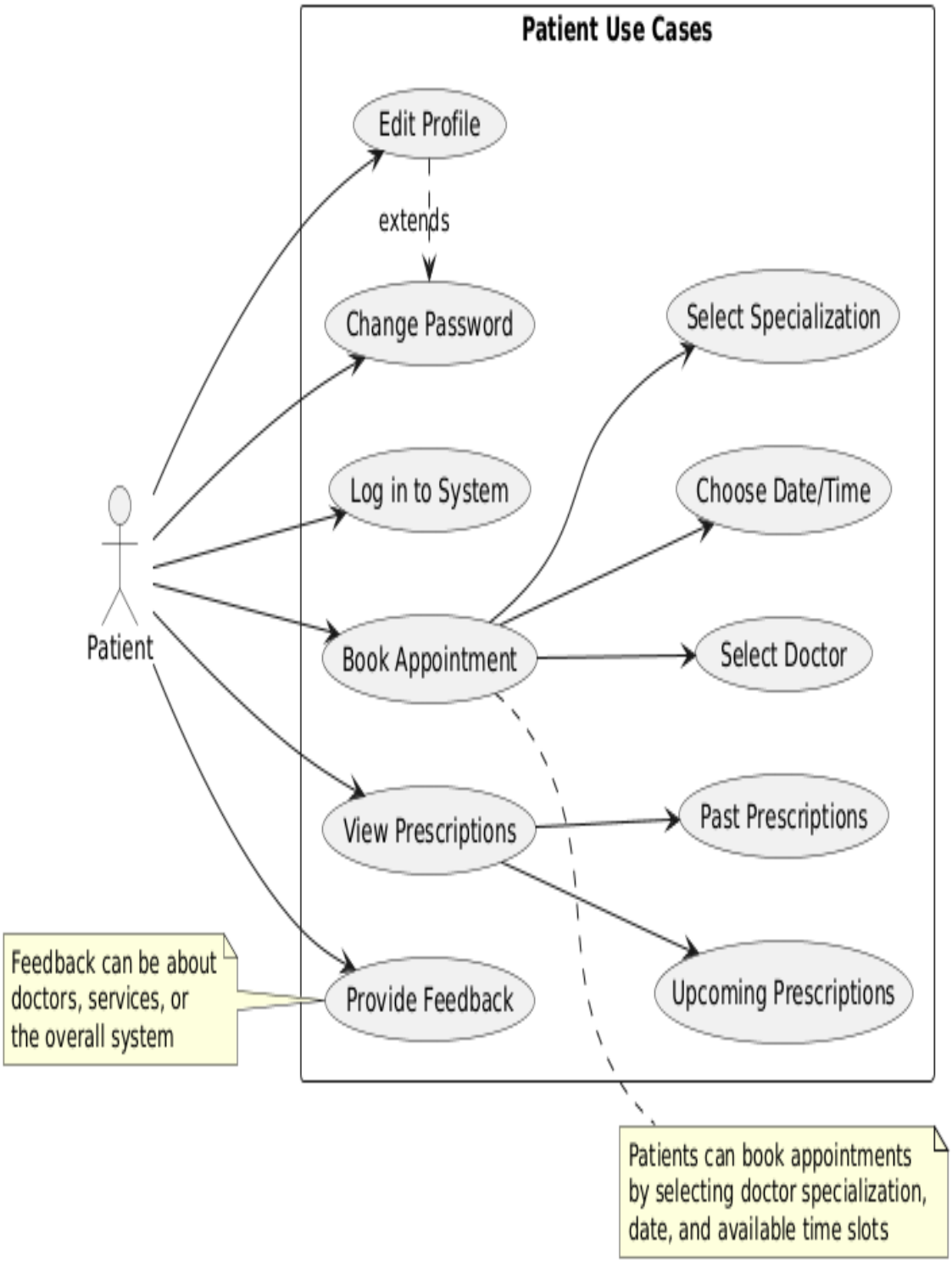


Figure: 3.2.3 **USE CASE DIAGRAM**

3.3 DESCRIBE THE EVENTS FLOW FOR USE CASE

The Basic Flow of Events specifies the interactions between the actor(s) and the system for the ideal case, where everything goes as planned, and the actor's goal (the observable result of value) is met. The basic flow represents the main capability provided by the system for this use case. The two main parts of the flow of events are **basic flow of events** and **alternative flows of events**. The alternative flows of events cover behavior of optional or exceptional character in relation to the normal behavior, and also variations of the normal behavior.

Registration:

Use case tables are a structured format for documenting use cases in text form. They can be a very simple list of basic information about use cases or contain more detailed information.

1. Use must be register to access the application.
2. After signup user must assure that valid Gmail, otherwise authentication message occurs.

Use Case ID:	**UC 01** **(Table.**
Use Case Name:	**Registration**
Actor:	User
Description:	User registers by providing email, password, and other required details
Trigger:	Click on the "Register" button.
Post conditions:	Before registration Email and password must be valid
Normal Flow:	• User open App • Select register new member. • Fill the form • Click on the "Register" button.
Alternative Flow	N/A
Exceptions:	

Table: 3.3. 1 REGISTRARTION

Login:

After the successfully registered user can login the application just enters the Gmail and password.

Before login Admin/Patient/Doctor must remember the user's name and password.

If user enter wrong password, the authentication message occurs.

Use Case ID:	UC 02 (Table.2
Use Case Name:	**Login**
Actor:	User
Description:	user can Login to the application
Trigger:	Click on the "Login" button.
Post conditions:	Valid and unique email; strong password New account is created and saved.
Normal Flow:	• Admin open app • Fill the Login form. • After successfully Login user can easily access to the Dashboa
Alternative Flows	N/A
Exceptions:	• User Name is required • Password is required

Table: 3.3. 2 LOGIN

Book Appointment

Use Case ID:	UC 03 (Table.03)
Use-Case Name:	**Book appointment**
Actor:	Patient
Description:	Patient books an appointment with a doctor
Trigger:	Click on "book appointment".
Post conditions:	Appointment is stored in the database and visible to doctor or patient.
Normal Flow:	• Patient logs in clicks "book appointment" • selects doctors, date, and time & save the appointment
Alternative Flows	N\A
Exceptions:	Doctor unavailable .time slot already booked .invalid input.

Table: 3.3.3 Book Appointment

View appointment history

Use Case ID:	UC 04 (Table.04)
Use Case Name:	**View appointment history**
Actor:	Patient
Description:	Patient views previous and upcoming appointment history
Trigger:	Click on “my appointment”
Post conditions:	List of appointment is displayed
Normal Flow:	• Patient open dashboard • Clicks “my appointment”

Table: 3.3. 4 View appointment history

View prescription:

Use Case ID:	UC 05 (Table.5)
Use Case Name:	**View prescription**
Actor:	Patient
Description:	Patient views prescription issued by doctor.
Trigger:	Click on “prescription”.
Post conditions:	Prescription details are displayed.
Normal Flow:	• Patient logs in click “prescription”. • system loads and displays prescription records.
Alternative Flows:	N/A
Exceptions:	• No prescriptions found.

Table: 3.3. 5 View Prescription

Manage Doctor:

Use Case ID:	UC 06 (Table.(
Use Case Name:	**Manage Doctors & Users**
Actor:	Admin
Description:	Admin can add, update, or remove doctors & Users.
Trigger:	Click on the "Wait for ME" check box
Post conditions:	Doctor list is updated.
Normal Flow:	• Admin opens "Manage doctors" Or "Manage Users". • click add/edit • system performs database operation.
Alternative Flow	N/A
Exceptions:	• invalid data. • deletion blocked due to active appointment

Table: 3.3. 5 Manage Doctor:

View appointment (doctors/ admin):

Use Case ID:	UC 07 (Table.7
Use Case Name:	**View appointment (doctors/ admin)**
Actor:	Admin
Description:	View list of appointment with details.
Trigger:	Click on the "appointment". with details.
Post conditions:	User must be login.
Normal Flow:	• Click on "appointment". • system fetches and displays appointment.
Alternative Flow	N/A
Exceptions:	• No data found. • server error.

Table: 3.3. 6 View appointment (doctors/ admin)

Diagnose patient:

Use Case ID:	UC 08 (Table.0
Use Case Name:	Diagnose patient
Actor:	Doctor
Description:	Doctors writes and saves patient diagnosis
Trigger:	Click on the “add diagnose”.
Post conditions:	Diagnosis added to patient history.
Normal Flow:	• Doctors open appointment details. • click “add diagnose” • fill in the diagnose
Alternative Flows:	N/A
Exceptions:	• Invalid data • from not submitted.

Table: 3.3. 8 Diagnose patient

Logout:

Use Case ID:	UC 09 (Table.0
Use Case Name:	Logout
Actor:	User
Description:	User can Logout to the web application
Trigger:	Click on the “Logout” button.
Post conditions:	Before Login Gmail and password must be valid.
Normal Flow:	• Admin/Doctor/Patient open side drawer. • Select “Logout” form Option. • After this process any user can easily logout the app
Alternative Flows:	N/A

Table: 3.3. 7 log Out

CHAPTER 4
DESIGN

4.1 ARCHITECTURE DIAGRAM

Architectural Diagram:

An architectural diagram is a diagram of a system that is used to abstract the overall outline of the software system and the relationships, constraints, and boundaries between components. It is an important tool as it provides an overall view of the physical deployment of the software system and its evolution roadmap.

Functions of an Architectural Diagram:

A diagram much like a picture is worth a thousand words. In other words, an architectural diagram must serve several different functions. To allow relevant users to understand a system architecture and follow it in their decision making, we need to communicate information about the architecture. Architectural diagrams provide a great way to do this. To put down some major functions, an architectural diagram needs to:

1. Break down communication barriers
2. Reach a consensus
3. Decrease ambiguity

The architecture diagram of "**Hospital Management System**" shows the relation between the technologies used, how the users interact with it, and how that leads to data being stored.

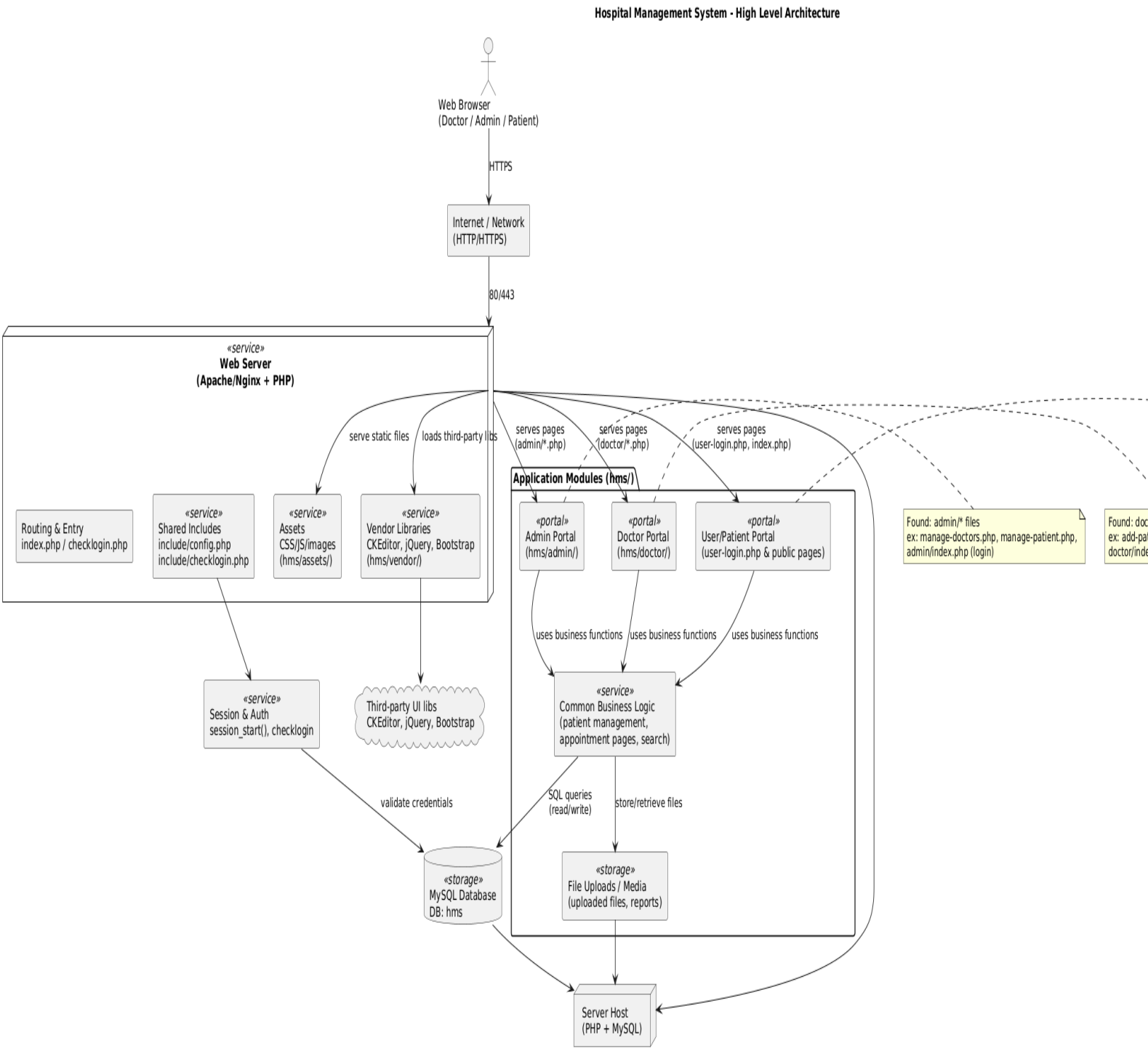


Figure: 4.1. 1 ARCHITECTURE DIAGRAM

4.2 ERD WITH DATA DICTIONARY

ER diagram:

An Entity Relationship (ER) Diagram is a type of flowchart that illustrates how "entities" such as people, objects or concepts relate to each other within a system. ER Diagrams are most often used to design or debug relational databases in the fields of software engineering, business information systems, education and research. Also known as ERDs or ER Models, they use a defined set of symbols such as rectangles, diamonds, ovals and connecting lines to depict the interconnectedness of entities, relationships and their attributes. They mirror grammatical structure, with entities as nouns and relationships as verbs.ER diagrams are related to data structure diagrams (DSDs), which focus on the relationships of elements within entities instead of relationships between entities themselves.

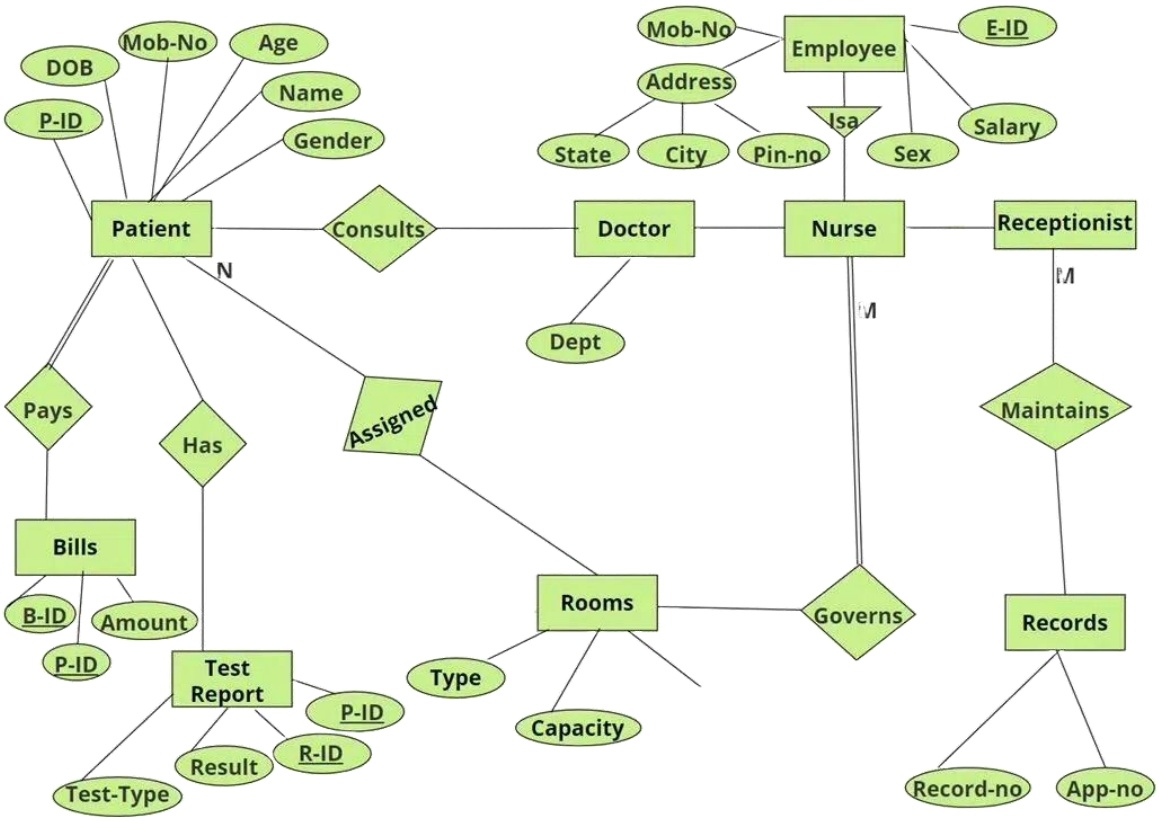


Figure: 4.1. 2 ERD WITH DATA DICTIONARY

4.3 DATA FLOW DIAGRAM (LEVEL 0 AND LEVEL 1)

Data flow diagram

A data flow diagram (DFD) maps out the flow of information for any process or system. It uses defined symbols like rectangles, circles and arrows, plus short text labels, to show data inputs, outputs, storage points and the routes between each destination. Data flowcharts can range from simple, even hand drawn process overviews, to in depth, multi-level DFDs that dig progressively deeper into how the data is handled. They can be used to analyze an existing system or model a new one. Like all the best diagrams and charts, a DFD can often visually "say" things that would be hard to explain in words, and they work for both technical and nontechnical audiences, from developer to CEO. That's why DFDs remain so popular after all these years. While they work well for data flow software and systems, they are less applicable nowadays to visualizing interactive, real time or database-oriented software or systems.

4.3.1 0 LEVEL DFD:

It is also known as a context diagram. It's designed to be an abstraction view, showing the system as a single process with its relationship to external entities. It represents the entire system as a single bubble with input and output data indicated by incoming/outgoing arrows.

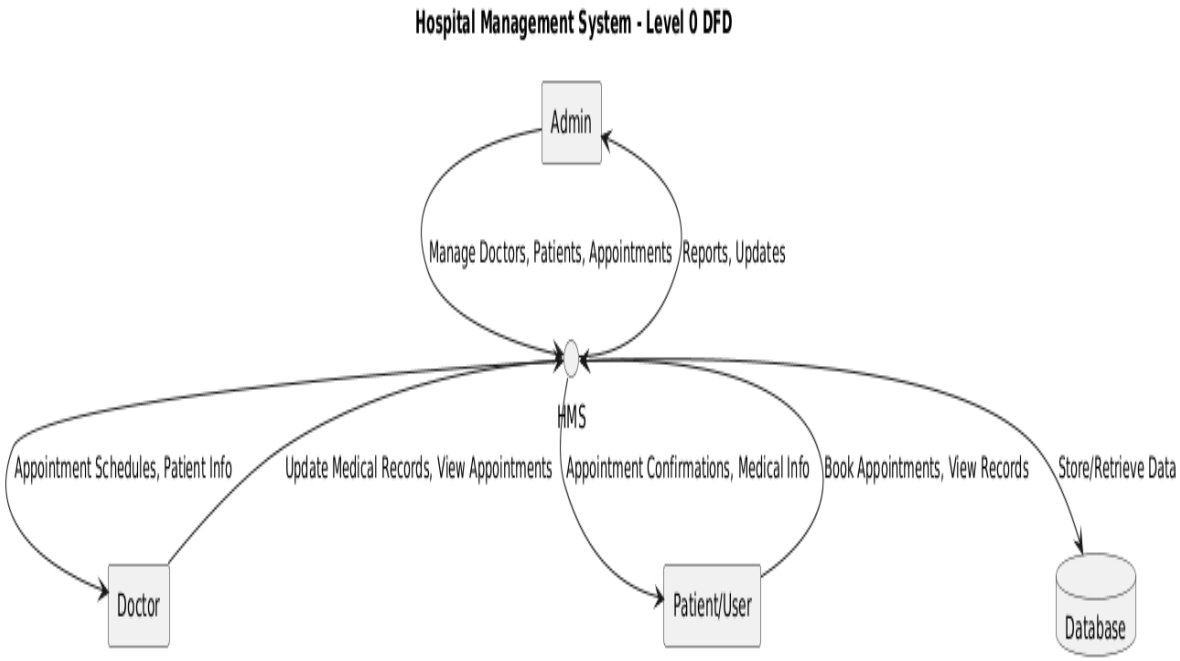


Figure: 4.1. 3 DATA FLOW DIAGRAM 0 LEVEL

4.3.2 1 LEVEL DFD:

In 1 level DFD, the context diagram is decomposed into multiple bubbles/processes. In this level, we highlight the main functions of the system and breakdown the high-level process of 0 level DFD into sub processes.

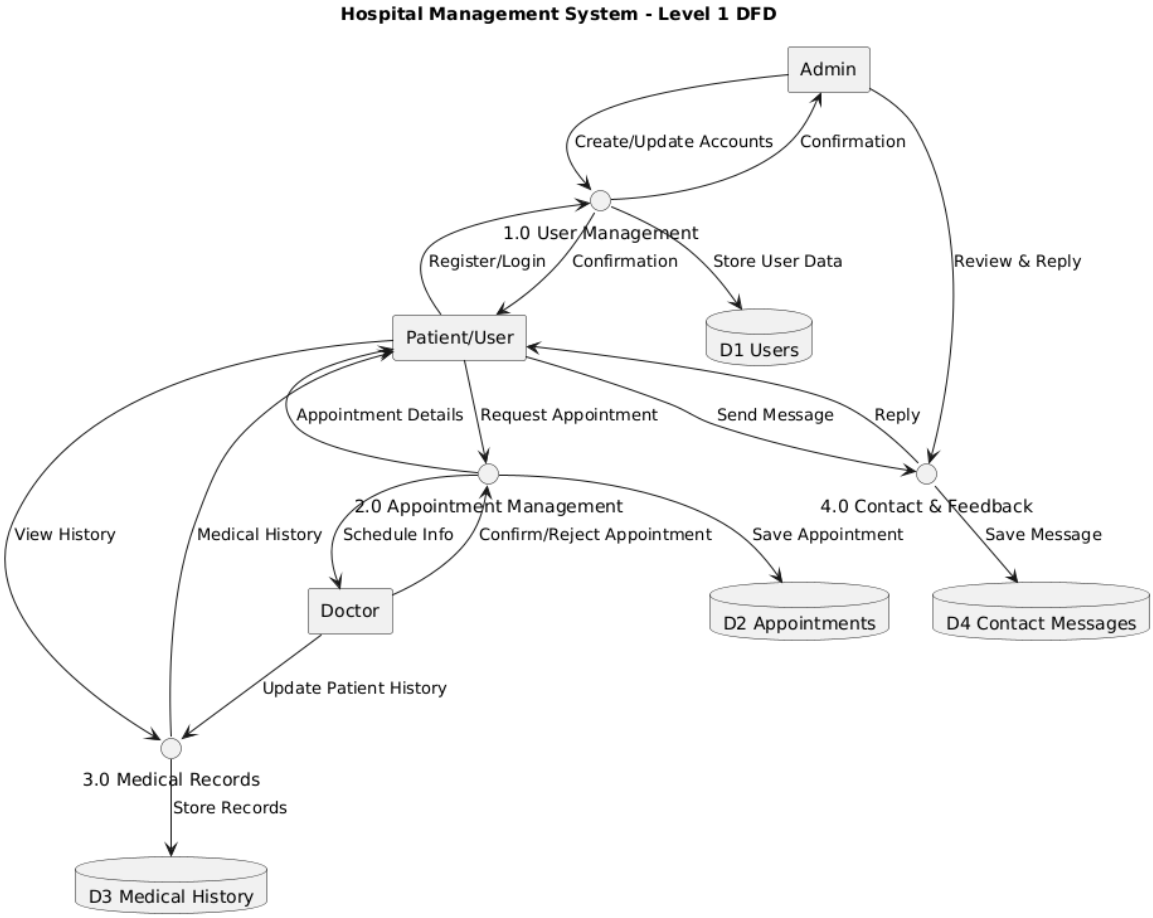


Figure: 4.1. 4 DATA FLOW DIAGRAM 1 LEVEL

1 Level DFD are still a gernal over view but they go into more detail than a context diagram. In level 1 DFD, the single process node context diagram is broken down into sub process as the process are added, the diagram will need addition data flows and data stores to link them together. 1 Level DFD are still a gernal over view but they go into more detail than a context diagram. In level 1 DFD, the single process node context diagram is broken down into sub process as the process are added, the diagram will need addition data flows and data stores to link them together.

4.4 CLASS DIAGRAM

Class diagrams are the most common diagrams used in UML. Class diagram consists of classes, interfaces, associations, and collaboration. Class diagrams basically represent the object-oriented view of a system, which is static in nature.

Active class is used in a class diagram to represent the concurrency of the system.

A class notation consists of three parts:

1. Class name: The name of the class appears in the first partition.
2. Class Attributes: Attributes are shown in the second partition.
3. Class Operations (Methods): Operations are shown in the third partition. The below diagram has the following classes.
4. User (Admin, Doctor and Patient)
5. Doctor Specialization
6. Appointment
7. Medical History
8. User-Log

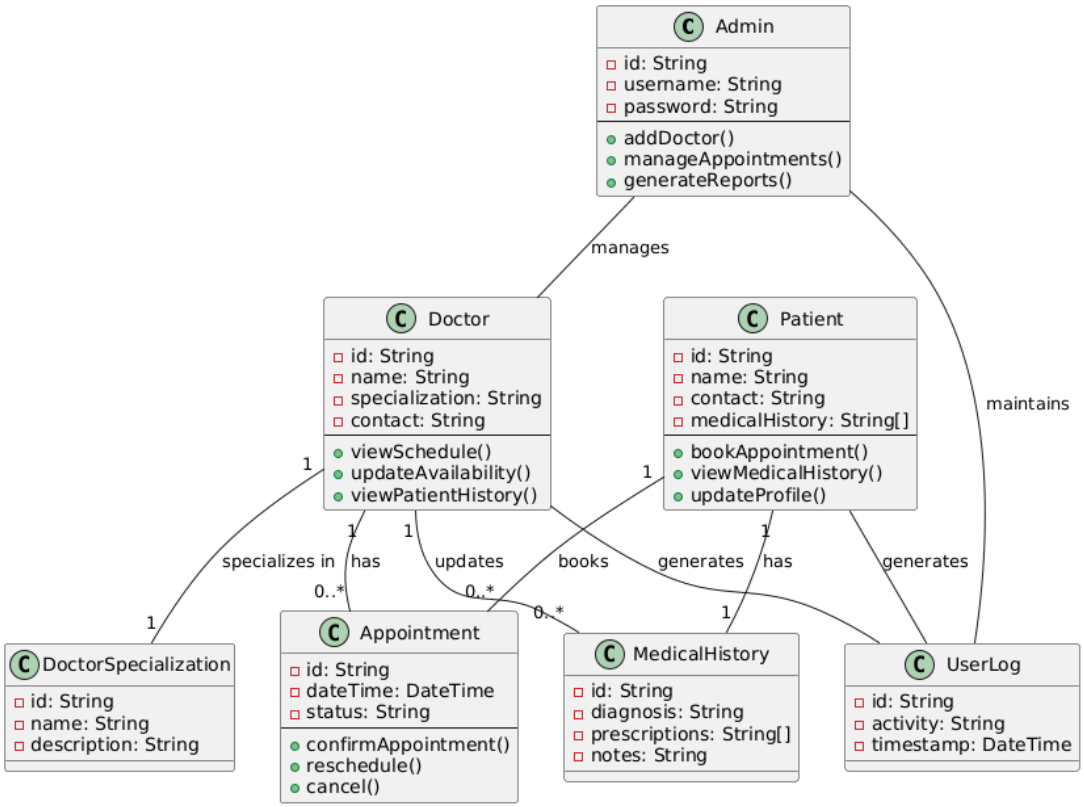


Figure: 4.1. 5 CLASS DIAGRAM

4.5 OBJECT DIAGRAM

Object diagrams can be described as an instance of class diagram. Thus, these diagrams are more close to real life scenarios where we implement a system.

It has the following objects.

i. Admin1
ii. User1
iii. Doctor1
iv. Patient1
v. Appointment1

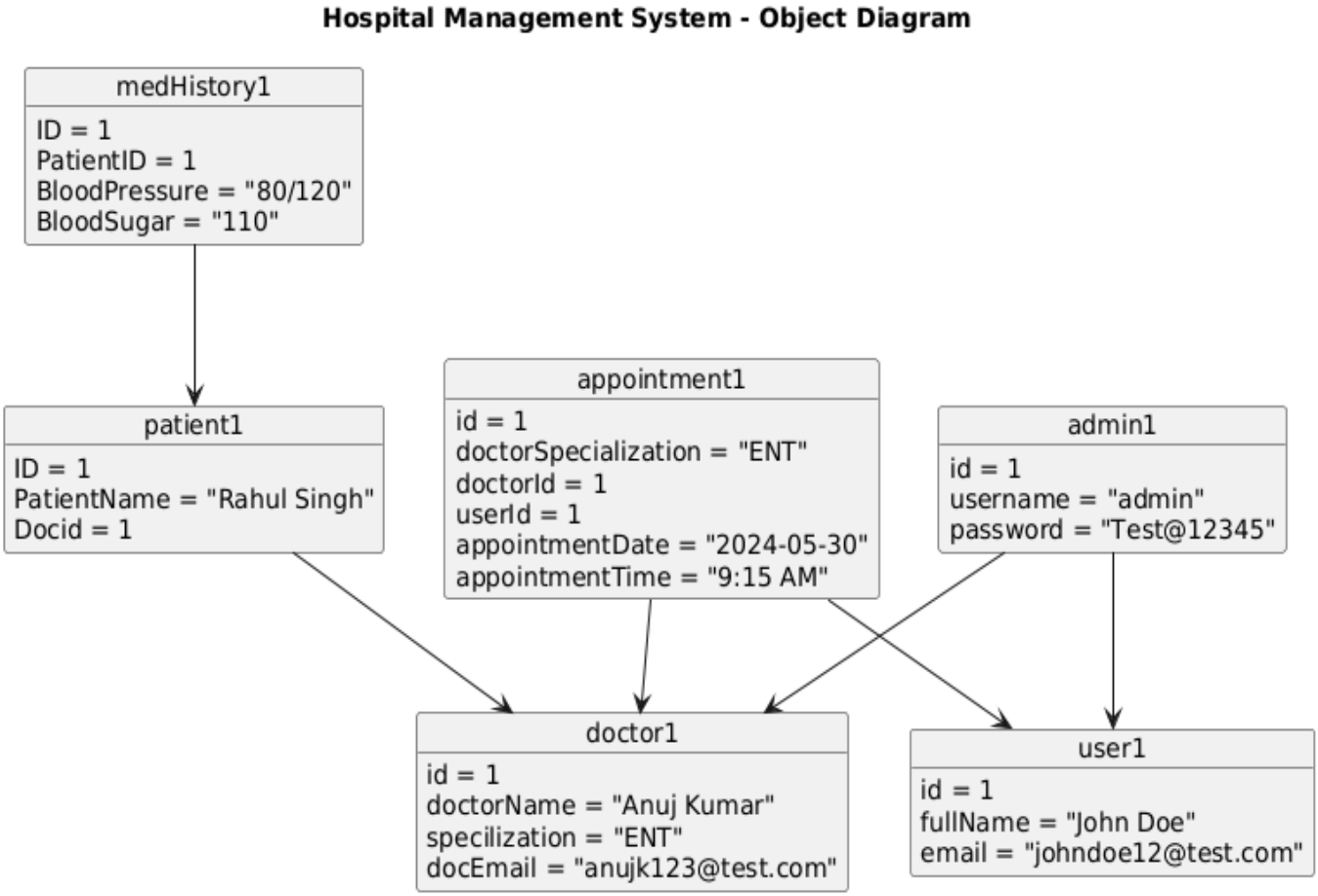


Figure: 4.1.6 OBJECT DIAGRAM

4.6 SEQUENCE DIAGRAM

A sequence diagram is a Unified Modeling Language (UML) diagram that illustrates the sequence of messages between objects in an interaction. A sequence diagram consists of a students of objects that are represented by lifelines, and the messages that they exchange over time during the interaction.

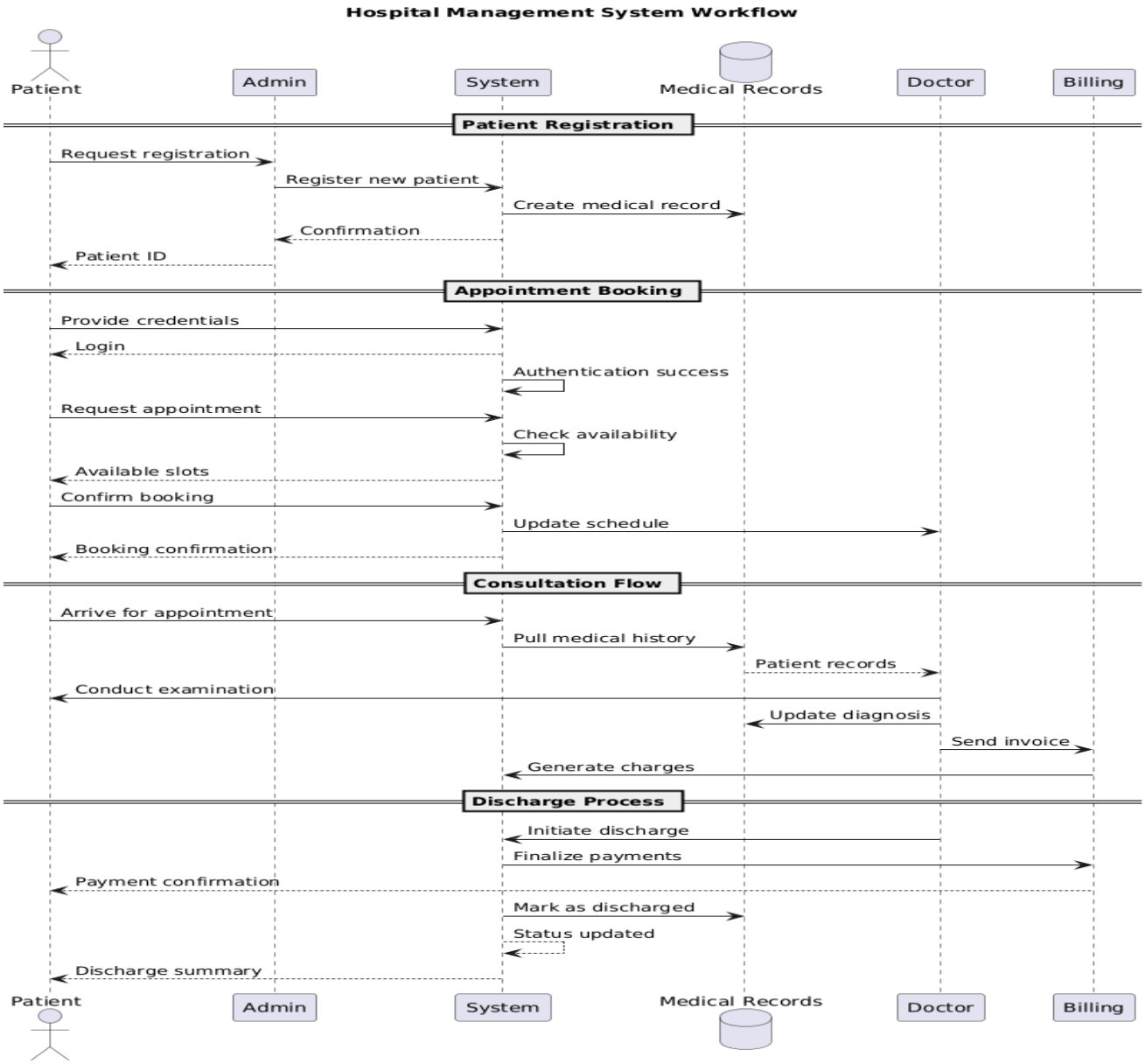


Figure: 4.1. 7 SEQUENCE DIAGRAM

4.7 ACTIVITY DIAGRAM

An activity diagram visually presents a series of actions or flow of control in a system similar to a flowchart or a data flow diagram. Activity diagrams are often used in business process modeling. They can also describe the steps in a use case diagram. Activities modeled can be sequential and concurrent.We use the activity diagram for "**Hospital Management System**" for explain the objects:

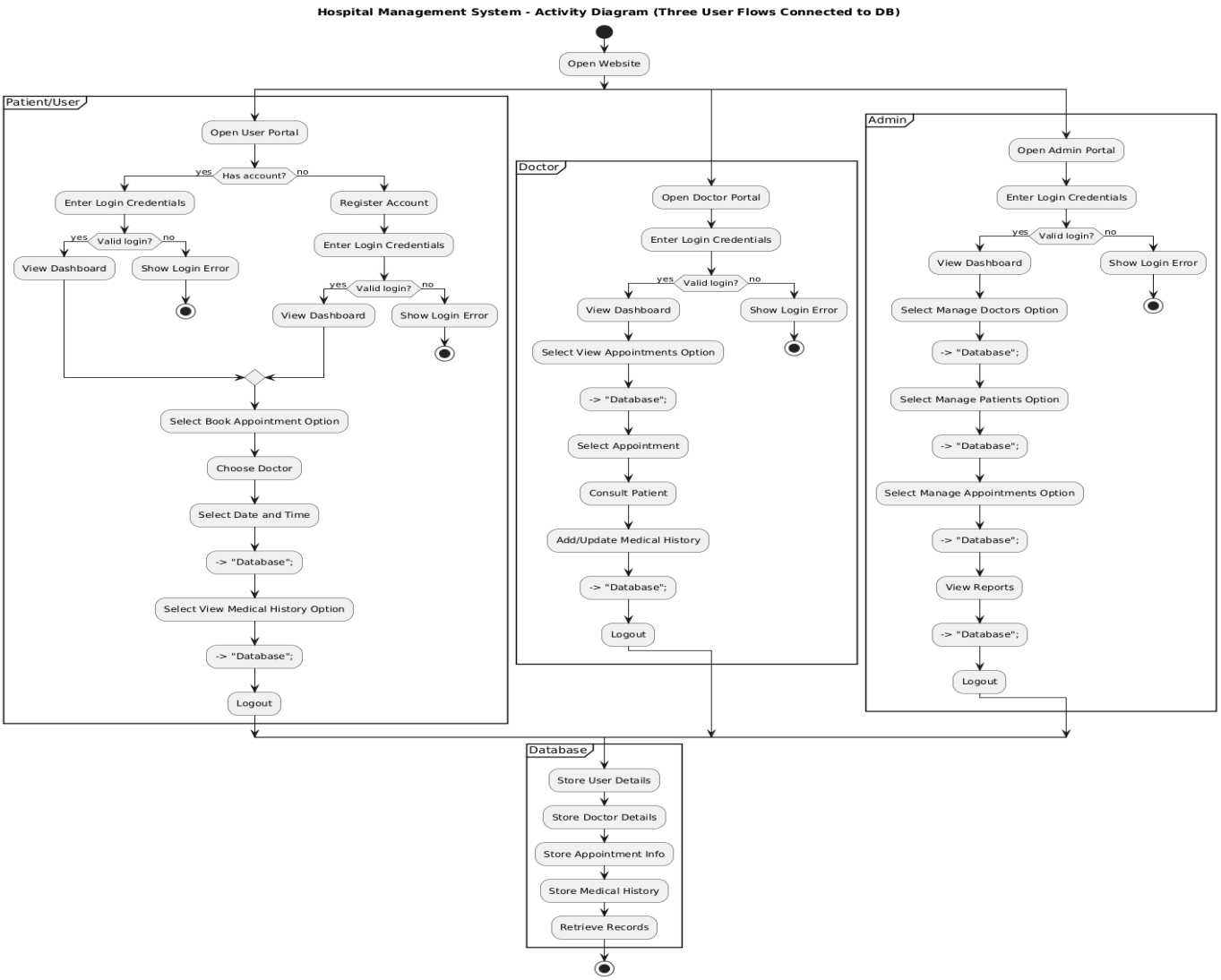


Figure: 4.1. 8 ACTIVITY Diagram

4.8 COLLABORATION DIAGRAM

Collaboration diagram is another form of interaction diagram. It represents the structural organization of a system and the messages sent/received. Structural organization consists of objects and links.

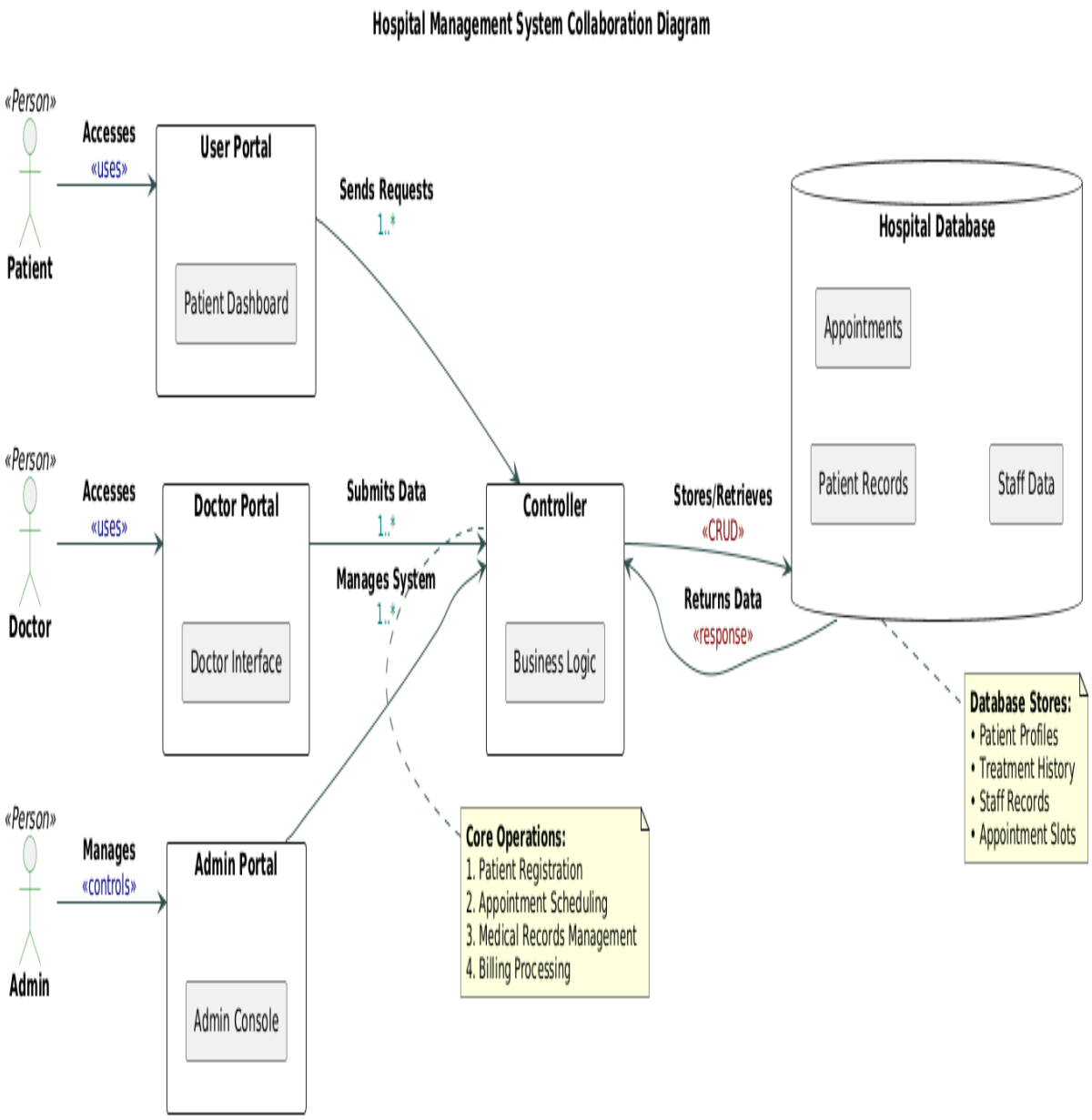


Figure: 4.1. 9 COLLABORATION DIAGRAM

4.9 STATE TRANSITION DIAGRAM

State transition diagrams describe all of the states that an object can have, the events under which an object changes state (transitions), the conditions that must be fulfilled before the transition will occur (guards), and the activities undertaken during the life of an object (actions). State transition diagrams are very useful for describing the behavior of individual objects over the full set of use cases that affect those objects. State transition diagrams are not useful for describing the collaboration between objects that cause the transitions. For those not familiar with the notation used for state transition diagrams, some explanation is in order.

1. State. A condition during the life of an object in which it satisfies some condition, performs some action, or waits for some event.
2. Event. An occurrence that may trigger a state transition. Event types include an explicit signal from outside the system, an invocation from inside the system, the passage of a designated period of time, or a designated condition becoming true.
3. Guard. A Boolean expression which, if true, enables an event to cause a transition.
4. Transition. The change of state within an object.
5. Action. One or more actions taken by an object in response to a state change.

Take a look at the Activity Diagram which models the flow of actions for an incident. Such an Activity Diagram focuses on the flow of data within a system.

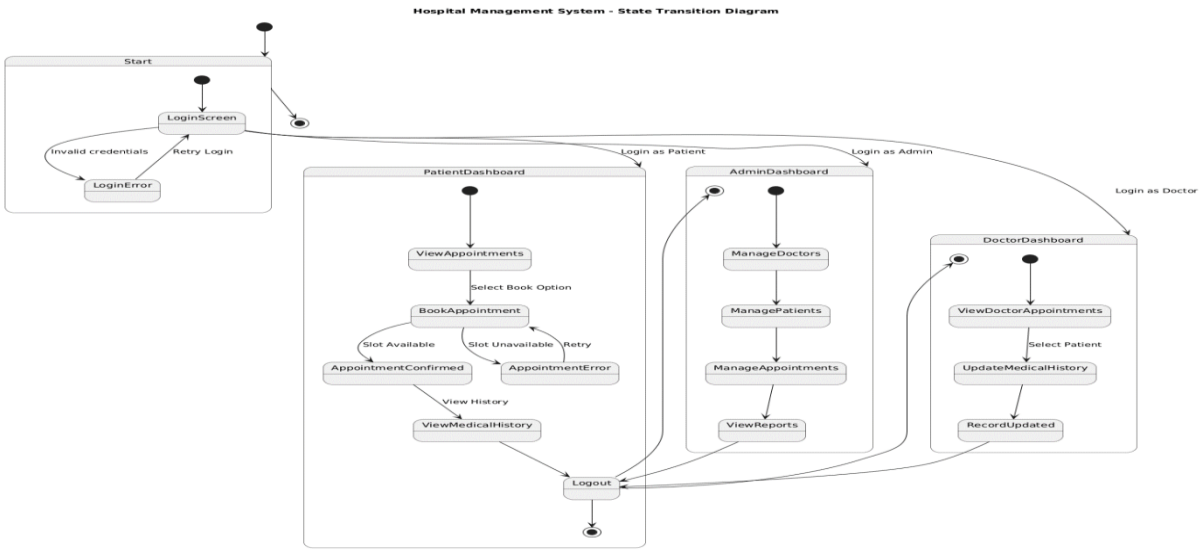


Figure: 4.1. 10 STATE TRANSITION DIAGRAM

CHAPTER 5

IMPLEMENTATION

After the workflow process and the proposed systems have been described, the implementation model takes place which is given as follows.

5.1 COMPONENT DIAGRAM

A component diagram models the physical aspects of a system, such as executables, libraries, files, and documents. It illustrates the organization and dependencies among system components. The Component Diagram for the **Hospital Management System** depicts the components, provided and required interfaces, ports, and relationships between key actors like Admin, Doctors, Patients. This diagram is essential in Component-Based Development **(CBD)** and Service-Oriented Architecture **(SOA)** to represent modular services and their interactions. For the evaluation and management of the FYP, this UML diagram clarifies the structure of physical components (e.g., patient registration module, appointment scheduler, billing system) and their interconnections, ensuring efficient system design and scalability.

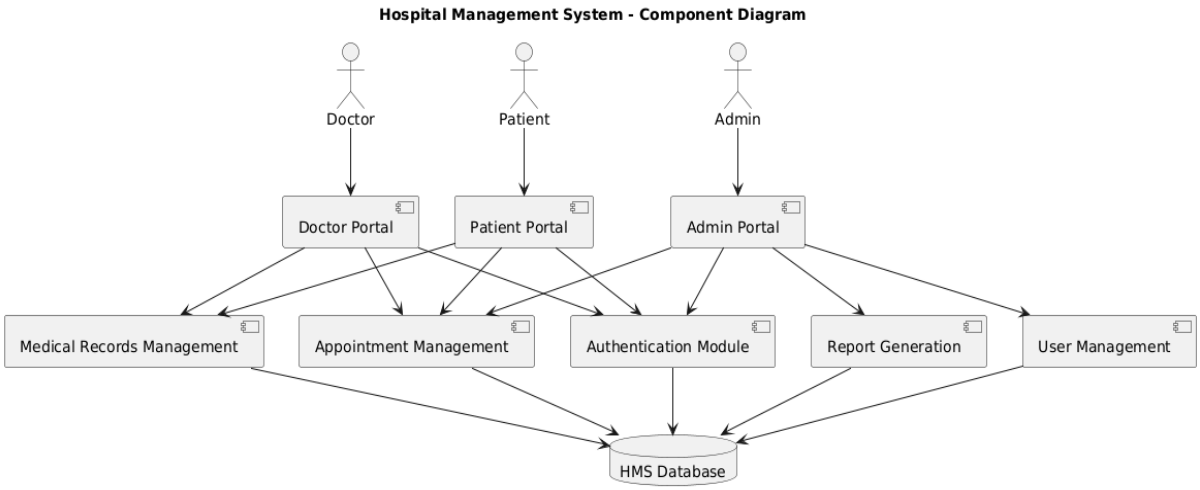


Figure: 5.1. 1 Component Diagram

5.2 DEPLOYMENT DIAGRAM

Deployment diagrams consist of **nodes** (physical or virtual hardware) and their connections, illustrating how software components are deployed across the system's infrastructure. These diagrams are critical for visualizing the deployment architecture and are primarily used by IT deployment team**s** to ensure efficient system setup and scalability.In the context of a **Hospital Management System**, a UML deployment diagram models the physical deployment of software artifacts (e.g., databases, web servers, and applications) across hardware nodes such as:

1. **Servers**: Hosting the central database (e.g., patient records, appointments).
2. **Workstations**: Used by doctors, nurses, and administrative staff to access the system.
3. **Mobile Devices**: For remote access by patients (e.g., appointment booking via apps).
4. **Cloud Systems**: Storing backups or enabling telemedicine services.

The diagram also highlights communication protocols (e.g., HTTPS for secure data transfer) and dependencies between nodes (e.g., a workstation connecting to the hospital's main server). This ensures compliance with healthcare data standards (like **HIPAA** or **GDPR**) while optimizing performance and reliability.

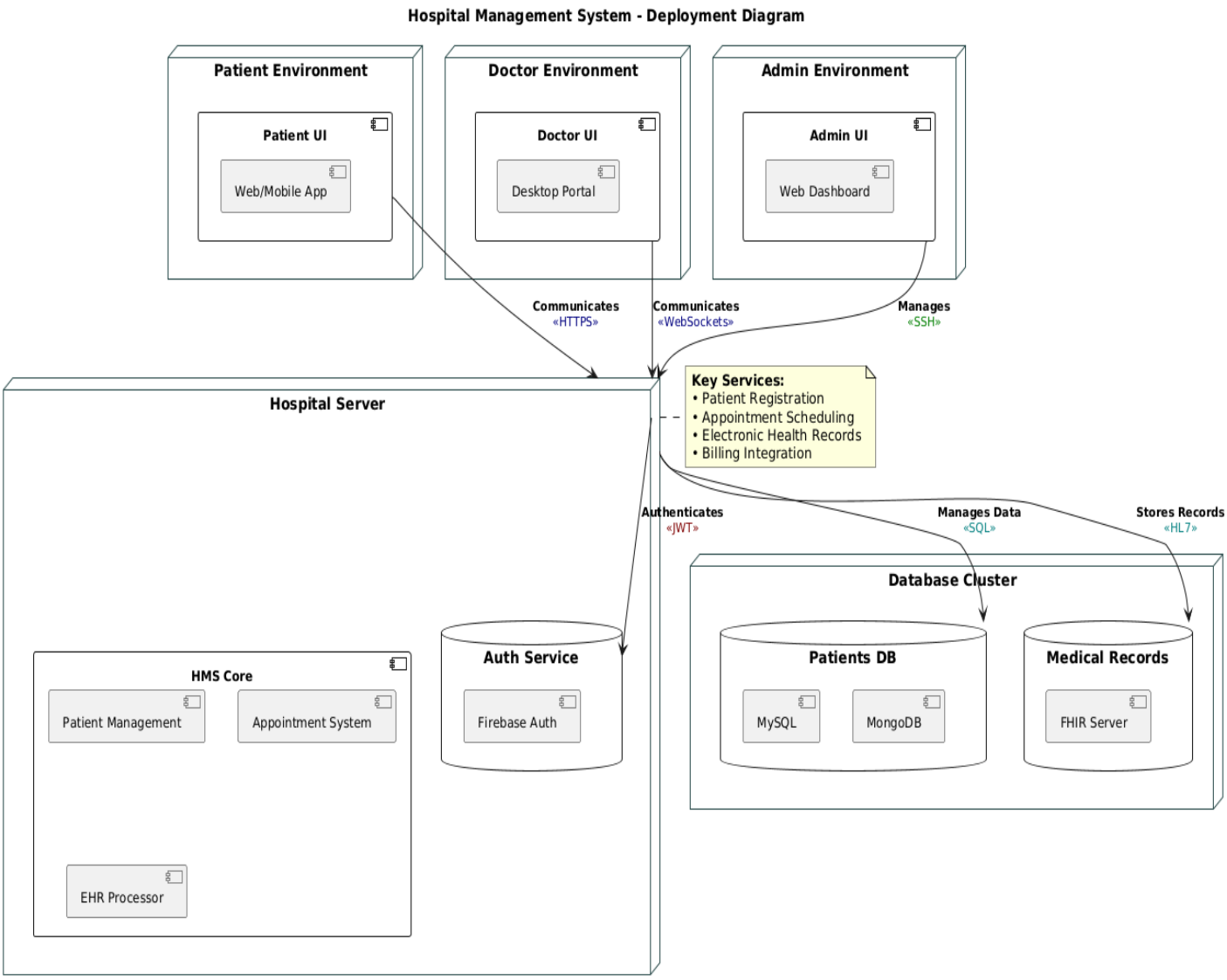


Figure: 5.2. 1 Deployment Diagram

CHAPTER 6

TESTING (SOFTWARE QUALITY ATTRIBUTES)

Testing:

Testing is the process of detecting errors. Testing performs a very critical role for quality assurance and for ensuring the reliability of software. The results of testing are used later on during maintenance also.

Testing Objectives:

1. To identify and reveal as many errors as possible in the tested software.
2. To bring the tested software, after correction of the identified errors and retesting, to an acceptable level of quality.
3. To perform the required tests efficiently and effectively, within budgetary and scheduling limitations.

Indirect objective:

1. To compile a record of software errors for use in error prevention (by corrective and preventive actions).

6.1 TEST CASE SPECIFICATION

Introduction:

There are some critical points, which they are needs to be tested by our project supervisor, and ourselves, end users.

There are some critical points, which have been tested:

1. Registration process
2. Login process
3. Logout process
4. Management process
5. Front end design test

Test Items:

1. Registration process
2. Login process

Features to be tested:

1. Registration process
2. Login process

Identify all features and combinations of features to be tested. Identify the test design specification associated with each feature and each combination of features.

Approach:

In section (features to be tested), we list down all the features that are necessary for Application. First of all, Administrator must be register. Administrator must be logged.

Item Pass/Fail Criteria:

When item has completed its purpose, item is passing otherwise, it is failing. It means item has not been matched the passing criteria.

Test Deliverables:

Test input data and test output data should be identified as deliverables.

In this section, registration forms, login form. These are the core features & deliverables.

Testing Tasks:

Selecting processing is a process, which depends upon two or more than two processes

Approvals:

Specify the names and titles of all persons who must approve this plan. Provide space for the signatures and dates.

6.1.1 FUNCTIONAL TESTING

This is a type of black box testing that is based on the specifications of the software that is to be tested.

There are five steps that are involved while testing an application for functionality.

1. The determination of the functionality that the intended application is meant to perform.
2. The creation of test data based on the specifications of the application.
3. The output based on the test data and the specifications of the application.
4. The writing of test scenarios and the execution of test cases.
5. The comparison of actual and expected results based on the executed test cases.

6.1.2 UNIT TESTING

Unit testing for the **Hospital Management System (HMS)** will validate critical functionalities to ensure reliability, security, and seamless healthcare workflows. Below are the key testing scenarios:

1. **Authentication & Role-Based Access**
   i. Verify **Firebase authentication** for **patients, doctors, nurses, and admins**, ensuring secure login/signup with email, password, and role-specific permissions.
   ii. Test **multi-factor authentication (MFA)** for sensitive roles (e.g., doctors accessing patient records).

2. **Patient Registration & Profile Management**
   i. Validate patient registration with required fields (name, contact, medical history, insurance details).
   ii. Test **duplicate record prevention** and data validation (e.g., valid phone number formats).

3. **Appointment Scheduling**
   i. Ensure patients can **book, reschedule, or cancel appointments** without conflicts.
   ii. Verify **real-time notifications** to doctors and patients (SMS/email).
   iii. Test **automated reminders** for upcoming appointments.

4. **Electronic Health Records (EHR) Management**
   i. Validate **doctor access** to assigned patient records (HIPAA compliance).
   ii. Test **real-time updates** (e.g., lab results, prescriptions) and audit logs for modifications.

5. **Telemedicine Integration**
   i. Verify **video consultation** stability and secure data encryption.

ii. Test **prescription e-signing** and sharing with pharmacies.

6. Emergency & Alert Systems

i. Validate **emergency alerts** (e.g., critical lab results trigger instant doctor notifications).

ii. Test **patient panic button** for immediate staff response.

7. Billing & Insurance Processing

i. Ensure **automated invoice generation** post-consultation.

ii. Test **insurance claim integration** (e.g., eligibility checks, claim submissions).

8. Admin & Staff Controls

i. Verify **admin privileges** (e.g., assigning doctors to departments, managing staff roles).

ii. Test **inventory alerts** for low medical supplies.

9. Integration Testing

i. Validate **third-party integrations** (e.g., lab systems, pharmacy databases, insurance portals).

ii. Ensure **API reliability** for external EHR systems (FHIR/HL7 standards).

10. Security & Compliance

i. Test **data encryption** for patient records (HIPAA/GDPR).

ii. Validate **session timeout** and unauthorized access prevention.

6.1.3 INTEGRATION TESTING

Integration testing for our **Hospital Management System (HMS)** will verify seamless interaction between all critical components and modules, ensuring a cohesive and error-free healthcare delivery system. This testing phase will validate that all integrated systems function together as designed, maintaining data integrity and providing optimal patient care.

Integration Test Scenarios:

1. **Authentication & User Management Integration**
    1. Verify integration between **Apache authentication** and role-based access control for patients, doctors, nurses, and administrators
    2. Test end-to-end registration flow including:
        i. Patient registration with medical history integration
        ii. Staff onboarding with department assignment
        iii. Multi-factor authentication for sensitive roles

2. **Appointment Scheduling System**
    1. Test integration between:
        i. Patient portal and doctor calendars
        ii. Notification system (SMS/email) for appointment confirmations

3. **Electronic Health Records (EHR) Integration**
    1. Verify real-time synchronization between:
        i. Doctor notes and patient records
        ii. Lab systems and diagnostic report
        iii. Pharmacy systems for e-prescriptions

4. **Security & Compliance Verification:**

1. Test HIPAA/GDPR compliance across all integrated systems
2. Validate encryption for all data transmissions
3. Verify audit trail integration across modules

5. **Performance Testing:**

1. Test system behavior under peak loads (e.g., multiple simultaneous emergencies)
2. Verify response times for critical integrations (e.g., emergency alerts)
3. Validate data synchronization speeds across distributed systems

This comprehensive integration testing approach ensures all HMS components work together seamlessly, providing reliable healthcare services while maintaining strict data security and compliance standards. The tests will verify both functional integration and non-functional requirements including performance, security, and usability across the complete system.

6.1.4 SYSTEM TESTING

System testing for our **Hospital Management System (HMS)** involves a comprehensive evaluation of the entire application to validate that all integrated components function cohesively in real-world healthcare scenarios, ensuring reliability, security, and optimal patient care delivery.

Testing Areas:

1. **Authentication & User Management**
    a. Verify **Firebase authentication** for all roles (patients, doctors, nurses, administrators)
    b. Test registration workflows including:
        i. Patient enrollment with medical history integration
        ii. Staff onboarding with department assignment
    c. Validate password recovery and role-based access control
    d. Ensure prevention of duplicate registrations

2. **Appointment Management**
    a. Test end-to-end appointment scheduling:
        i. Patient self-booking via portal
        ii. Doctor calendar synchronization
        iii. Automated SMS/email notifications
    b. Validate conflict detection for overlapping appointments
    c. Test cancellation/rescheduling workflows

3. **Electronic Health Records (EHR) System**
    a. Verify complete patient record management:
        i. Doctor notes and treatment plans
        ii. Lab result integration
        iii. Prescription generation
    b. Test audit trails for all record modifications
    c. Validate HIPAA-compliant data access controls

4. **Real-Time Patient Monitoring**
    a. Test integration of:
        i. Vital sign monitoring equipment
        ii. Nurse station dashboards
        iii. Emergency alert systems
    b. Validate threshold-based notifications (e.g., critical BP levels)

5. **Emergency Response System**
    a. Validate:
        i. Emergency button functionality
        ii. On-call doctor notification system
        iii. Critical alert escalation protocols

Non-Functional Testing:

1. **Performance Testing**
    a. Evaluate system response under peak loads (e.g., multiple emergency cases)
    b. Test data synchronization speeds across distributed systems
    c. Verify real-time monitoring update intervals

2. **Security Testing**
    a. Validate end-to-end encryption for all data transmissions
    b. Test HIPAA/GDPR compliance
    c. Verify protection against common vulnerabilities (OWASP Top 10)

3. **Usability Testing**
    a. Evaluate interface intuitiveness for all user roles
    b. Test accessibility compliance (WCAG)
    c. Validate multilingual support

4. **Reliability Testing**
    a. Verify system uptime during critical operations
    b. Test failover mechanisms
    c. Validate backup and recovery procedures

Expected Outcomes:

1. A fully validated HMS that ensures:
    a. Accurate patient data management
    b. Efficient clinical workflows
    c. Secure information handling

This comprehensive system testing approach will deliver a robust Hospital Management System that meets all functional requirements while adhering to healthcare industry standards and regulations. The testing will verify both technical performance and clinical usability, ensuring the system enhances rather than disrupts healthcare delivery.

6.1.5 REGRESSION TESTING

Regression testing for our **Hospital Management System (HMS)** will systematically verify that recent system updates and modifications have not adversely impacted existing critical functionalities. This comprehensive retesting process ensures continuity of care and system reliability across all clinical and administrative operations.

Core Regression Test Areas:

1. **Authentication & Security**
    a. Revalidate Firebase authentication workflows for all user roles (patients, doctors, administrators)
    b. Confirm unchanged functionality of:
        i. Multi-factor authentication protocols
        ii. Password recovery processes
        iii. Role-based access controls
    c. Verify HIPAA compliance remains intact after updates

2. **Patient Management**
    a. Retest patient registration with complete medical history integration
    b. Validate unaltered functionality of:
        i. Duplicate record prevention
        ii. Insurance eligibility verification
        iii. Emergency contact management

3. **Clinical Workflows**
    a. Re-execute tests for:
        i. Appointment scheduling/rescheduling flows
        ii. Electronic health record (EHR) updates
        iii. Prescription generation and transmission
    b. Confirm lab result integration remains accurate

4. **Critical Care Systems**
    a. Revalidate emergency response features:
        i. Code blue alert systems
        ii. Critical value notifications
        iii. Rapid response team paging
    b. Retest medication administration workflows

5. **Billing & Financials**
    a. Verify unchanged functionality of:
        i. Automated charge capture
        ii. Insurance claim processing
        iii. Patient statement generation
    b. Reconcile financial reports with treatment records

6. **System Integration**
    a. Retest interoperability with:
        i. Pharmacy systems
        ii. Laboratory information systems
        iii. Imaging/PACS systems
    b. Confirm HL7/FHIR compliance

Non-Functional Regression Testing:

1. **Performance**
    i. Reassess system response times under load
    ii. Verify unchanged data synchronization speeds
    iii. Confirm real-time monitoring intervals

2. **Security**
    a. Retest:
        i. Data encryption standards
        ii. Audit trail completeness

3. **Compliance**
    a. Reverify:
        i. HIPAA security rules
        ii. GDPR requirements
        iii. Local healthcare regulations

Testing Approach:

i. Automated test suites for high-frequency workflows
ii. Manual testing for complex clinical scenarios
iii. Production data validation in staging environment
iv. Edge case verification for critical care functions

Expected Outcomes:

I. Confirmation that system updates maintain:
    a. Patient safety standards
    b. Clinical workflow integrity
    c. Data security compliance
    d. Financial transaction accuracy
II. Documentation of any regression defects for immediate remediation

This rigorous regression testing protocol ensures the HMS continues to deliver reliable, compliant healthcare services while accommodating system enhancements and modifications. The process safeguards against unintended consequences of changes, maintaining the highest standards of patient care and operational efficiency.

6.1.6 ACCEPTANCE TESTING

Acceptance testing for our **Hospital Management System (HMS)** will validate that the system meets all clinical, operational, and regulatory requirements while delivering an optimal experience for patients, healthcare providers, and administrators. This testing phase ensures the HMS is ready for production deployment.

Test Scenarios:

1. **User Authentication & Role-Based Access**
    a. Verify seamless **registration/login** for:
        i. **Patients** (via email/phone with medical history capture)
        ii. **Doctors/Nurses** (with role-specific privileges)
        iii. **Administrators** (with system-wide access controls)

2. **Patient Management**
    a. Test end-to-end **patient onboarding**:
        i. Demographic data capture
        ii. Insurance verification
        iii. Consent form management (HIPAA/GDPR)
    b. Validate **duplicate record prevention** and data accuracy.

3. **Appointment Scheduling**
    a. Confirm:
        i. Real-time **doctor availability** displays
        ii. Automated **SMS/email reminders**
        iii. **Rescheduling/cancellation** workflows
    b. Test **waitlist management** for urgent cases.

4. **Clinical Workflows**
    a. Verify:
        i. **EHR integration** (lab results, prescriptions, treatment plans)
        ii. **Doctor-patient messaging** with read receipts
        iii. **E-prescription** transmission to pharmacies
    b. Validate **audit trails** for all clinical actions.

User Experience Validation:

1) **Patients**:
   a) Intuitive **self-service portal** for appointments/records
   b) Clear **medication instructions** and follow-up alerts
2) **Doctors**:
   a) Efficient **patient lookup** and charting tools
   b) Customizable **clinical templates**
3) **Administrators**:
   a) Comprehensive **staff management** controls
   b) Real-time **bed occupancy/inventory** monitoring

Non-Functional Testing:

1. **Performance**:
   i. Test system response during **peak loads** (e.g., morning rush hours)
   ii. Verify **data sync speeds** for critical updates (e.g., emergency alerts)

2. **Security**:
   i. Validate **end-to-end encryption** for all PHI
   ii. Test **session timeout** and unauthorized access prevention

3. **Compliance**:
   i. Reverify **HIPAA/GDPR** requirements
   ii. Validate **disaster recovery** protocols

Sign-Off Criteria:

1) All critical **clinical workflows** function as specified
2) No **high-severity defects** in patient safety features
3) **Stakeholder approval** from:
   a) Clinical staff (doctors/nurses)
   b) IT/Compliance teams

6.1.7 ALPHA TESTING

Alpha testing for our **Hospital Management System (HMS)** focuses on identifying bugs and usability issues within the controlled development environment. Internal developers and QA testers rigorously evaluate core functionalities, including:

1) **User authentication** (patient/doctor/admin registration and login)
2) **Appointment scheduling** and calendar synchronization
3) **Electronic Health Record (EHR)** integration and data accuracy

Testers also assess the **telemedicine module**, **inventory management**, and **reporting dashboards** for performance and usability. This phase aims to uncover critical defects before advancing to beta testing with external stakeholders

6.1.8 BETA TESTING

Beta testing for our **Hospital Management System (HMS)** involves deploying the application to a limited group of external users—including patients, doctors, nurses, and administrators—to evaluate real-world performance. This phase identifies residual bugs and usability issues missed during internal testing.Testers will assess critical workflows:

1) **Role-based authentication** and access controls
2) **Appointment scheduling** and notification system
3) **EHR integration** and clinical data accuracy
4) **Emergency alert** responsiveness
5) **Billing and insurance** processing

Feedback will focus on:

1) System reliability under actual clinical workloads
2) Intuitiveness of interfaces for all user roles
3) Accuracy of critical alerts and data synchronization

6.1.9 NON-FUNCTIONAL TESTING

This section is based upon testing an application from its non-functional attributes. Nonfunctional testing involves testing a software from the requirements which are nonfunctional in nature but important such as performance, security, user interface, etc.Some of the important and commonly used nonfunctional testing types are discussed below.

6.1.9.1 PERFORMANCE TESTING

It is mostly used to identify any bottlenecks or performance issues rather than finding bugs in a software. There are different causes that contribute in lowering the performance of a software −

1) Network delay
2) Client-side processing
3) Database transaction processing

Performance testing is considered as one of the important and mandatory testing type in terms of the following aspects −

Speed (i.e. Response Time, data rendering and accessing)

1) Capacity
2) Stability
3) Scalability

6.1.9.2 LOAD TESTING

Load testing for our **Hospital Management System (HMS)** will rigorously evaluate system performance under both normal and peak operational conditions. This critical testing phase simulates high user concurrency to assess:

1) **Core System Metrics**
   a) Response times for key workflows (patient registration, appointment booking)
   b) Throughput during peak hours (e.g., morning outpatient rush)
   c) Server resource utilization (CPU, memory, database load)
2) **Critical Functionality Under Stress**
   a) Concurrent EHR access by multiple clinicians
   b) Bulk appointment scheduling during seasonal demand surges
   c) Emergency alert processing during mass casualty incidents

3) **Infrastructure Capacity**

a) Database performance with 10,000+ patient records
b) API response times for integrated systems (labs, pharmacies)
c) Session management during staff shift changes

4) **Failover Scenarios**

a) System behavior during network interruptions
b) Backup server activation times
c) Data synchronization recovery

Testing Methodology

1) Simulate 500+ concurrent users across roles (patients, doctors, admins)
2) Gradually increase load to identify breaking points
3) Monitor both frontend responsiveness and backend stability

Success Criteria:

1) Sub-2-second response for 95% of transactions
2) <1% error rate at projected peak loads
3) Graceful performance degradation (no complete failures)

This testing ensures the HMS maintains clinical-grade reliability during high-demand periods while protecting patient data integrity and care continuity.

6.1.9.3 STRESS TESTING

Stress testing for our **Hospital Management System (HMS)** will deliberately exceed normal operational limits to evaluate failure thresholds and recovery mechanisms. The testing will:

1) **Simulate Extreme Scenarios**

a) 5x anticipated patient registration volume
b) Mass concurrent EHR access during emergencies
c) System overload during peak admission periods
d) System performance during peak load.

2) **Target Critical Functions**

a) Real-time patient monitoring alerts
b) Emergency response system triggers
c) High-volume billing/insurance processing

3) **Monitor Failure Patterns**

a) Gradual performance degradation metrics
b) Data integrity during system crashes
c) Automated recovery success rates

4) **Key Evaluation Metrics**

a) Transaction failure rates under duress
b) Memory/CPU exhaustion thresholds
c) Database deadlock scenarios

5) **Testing Objectives**

a. Verify graceful service degradation (never complete failure)
b. Ensure emergency functions remain available
c. Validate data preservation during crashes

This rigorous evaluation guarantees the HMS maintains critical care functionality during disaster scenarios, protecting both patient safety and institutional operation.

6.1.9.4 USABILITY TESTING

Usability testing for our **Hospital Management System (HMS)** evaluates how intuitively patients, doctors, and administrators can navigate critical healthcare workflows. Test participants will perform key tasks including:

a. Patient registration and appointment booking
b. Doctor-patient messaging and e-prescription workflows
c. Emergency alert activation and response

We'll assess:

a. Clarity of medical data presentation
b. Efficiency of clinical workflows
c. Accessibility for users with varying tech literacy

Special focus will be given to:

a. EHR interface for quick patient lookups
b. Telemedicine session initiation
c. Critical alert visibility

The goal is to optimize the system for stress-free use in fast-paced healthcare environments, minimizing training needs while ensuring patient safety and efficient care delivery.

6.1.9.5 UI V/S USABILITY TESTING

UI testing involves testing the Graphical User Interface of the Software. UI testing ensures that the GUI functions according to the requirements and tested in terms of color, alignment, size, and other properties.On the other hand, usability testing ensures a good and user-friendly GUI that can be easily handled. UI testing can be considered as a sub part of usability testing.

6.1.9.6 SECURITY TESTING

Security testing for our **Hospital Management System (HMS)** rigorously evaluates protection against healthcare-specific cyber threats. Key focus areas include:

a. **Authentication safeguards** for patients, doctors and administrators
b. **Encryption** of sensitive data (PHI, treatment records, payment info)
c. Vulnerability checks for **SQL injection**, **XSS**, and **CSRF attacks**
d. **HIPAA/GDPR compliance** for data handling and access controls
e. **Role-based permissions** ensuring strict data segregation
f. **Audit trails** for all sensitive transactions
g. **Breach response** capabilities and recovery protocols

Testing validates the system's ability to maintain **patient confidentiality**, **data integrity**, and **service continuity** against evolving healthcare security threats.

6.1.9.7 PORTABILITY TESTING

Test Cases:

The Hospital management system was tested using unit, integration and system testing techniques. After every part of the system was implemented, it was tested using input to guarantee that each unit responded as expected.

Below are some of the tested cases that we are used to test the system:

Registration:

Table: 6.1. 1 REGISTRATION TEST CASE

Registration	**(Table. 1)**
Test Engineer:	Muhammad Arslan
Test Case ID:	TC1
Related UC/FR	UC1/FR1
Date:	02/Aug/2025
Purpose:	Verify successful patient registration with mandatory medical history field.
Pre Req:	User must have an email and other details to register.
Test Data:	As per form must require fields
Steps:	1. visit Register Page 2. enter Personal Details 3. enter Email 4. Enter Password 5. click Register
Status:	Pass

Table: 6.1.1 REGISTRATION TEST CASE

Login:

Login	**(Table. 2)**
Test Engineer:	Muhammad Arslan
Test Case ID:	TC2
Related UC/FR	UC2/FR2
Date:	02/Aug/2025
Purpose:	Need to ensure that user can only login successfully with its true Email and Password.
Pre Req:	User must be registered before Login
Test Data:	Email and Password.
Steps	Steps to carry out the test. 1. visit Login Page 2. enter Email 3. enter password 4. click login
Status:	Pass

Table: 6.1. 2 LOGIN TEST CASE

Add Patient:

Add Patient	**(Table. 3)**
Test Engineer:	Muhammad Arslan
Test Case ID:	TC3
Related UC/FR	UC3/FR3
Date:	02/Aug/2025
Purpose:	Add Patient
Pre Req:	User must be Login.
Test Data:	Add
Steps:	Steps to carry out the test. • Add Patient • Check Patient
Status:	Pass

Table: 6.1. 3 TEST CASE ADD PATIENT

Add Doctor:

Add Doctor	**(Table. 4)**
Test Engineer:	Muhammad Arslan
Test Case ID:	TC4
Related UC/FR	UC4/FR4
Date:	02/Aug/2025
Purpose:	Can add doctors
Pre Req:	User must be Login.
Test Data:	Add and check
Steps:	Steps to carry out the test. 1. Add Doctor 2. Check Doctor
Status:	Pass

Table: 6.1. 4 ADD DOCTOR TEST CASE

DELETE DOCTOR:

Delete Doctor	**(Table. 5)**
Test Engineer:	Muhammmad Arslan
Test Case ID:	TC7
Related UC/FR	UC7/FR7
Date:	02/Aug/2025
Purpose:	Can Delete Doctor.
Pre Req:	Delete and check
Test Data:	Steps to carry out the test. • Delete Doctor • Check Doctor
Steps:	User must be Login.
Status:	Pass

Appointment:

Appointment	**(Table. 6)**
Test Engineer:	Muhammad Arslan
Test Case ID:	TC5
Related UC/FR	UC5/FR5
Date:	02/Aug/2025
Purpose:	User can view all appointments on refresh
Pre Req:	User must be Login.
Test Data:	
Steps:	Steps to carry out the test. 1. Select appointment timing 2. View list
Status:	Pass

Table: 6.1. 5 APPOINTMENT TEST CASE

Manage Users:

Manage Users	**(Table. 7)**
Test Engineer:	Muhammad Arslan
Test Case ID:	TC6
Related UC/FR	UC6/FR6
Date:	02/Aug/2025
Purpose:	To manage all users.
Pre Req:	User must be Login.
Test Data:	
Status:	Pass

Table: 6.1. 6 Manage Users TEST CASE

Logout:

Logout	**(Table. 8)**
Test Engineer:	Muhammad Arslan
Test Case ID:	TC08
Related UC/FR	UC08/FR08
Date:	02/Aug/2025
Purpose:	After all process user can logout to the application.
Pre Req:	User can logout.
Test Data:	Null
Steps:	Steps to carry out the test. 1. Click Logout.
Status:	Pass

6.2 BLACK BOX TEST CASES

Black box testing is considered high level testing, which means that its main goal is to test functionalities from the behavioral point of view.

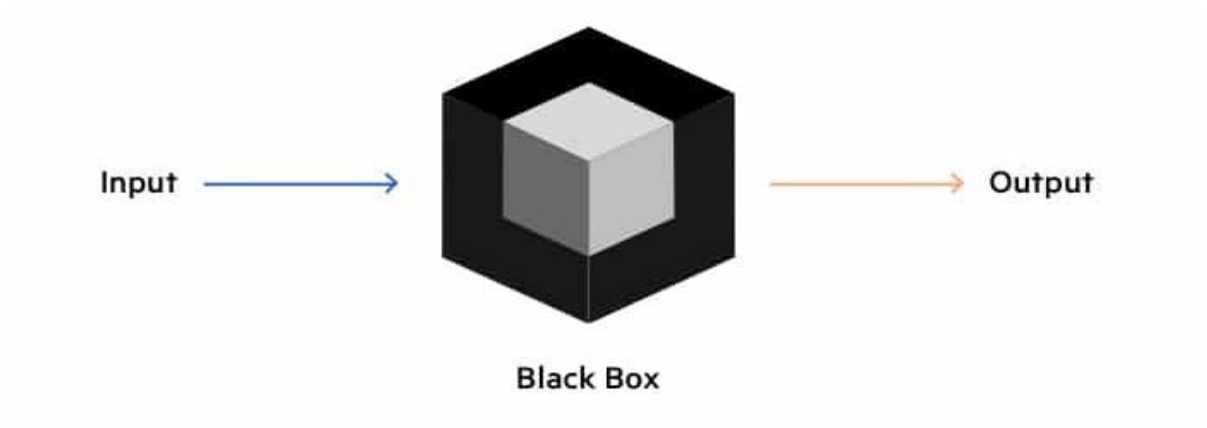


Figure: 6.2. 1 BLACK BOX TEST CASE

ADVANTAGES

1. Well suited and efficient for large code segments.
2. Code access is not required.
3. Clearly separates user's perspective from the developer's perspective through visibly defined roles.
4. Large numbers of moderately skilled testers can test the application with no knowledge of implementation, programming language, or operating systems.

DISADVANTAGES

1. Limited coverage, since only a selected number of test scenarios is actually performed.
2. Inefficient testing, due to the fact that the tester only has limited knowledge about an application.
3. Blind coverage, since the tester cannot target specific code segments or errorprone areas.
4. The test cases are difficult to design.

There are a set of approaches for black box testing:

1) **Manual UI Testing:** In this approach, a tester checks the system as a user. Check and verify the user data, error messages.
2) **Automated UI Testing:** In this approach, user interaction with the system is recorded to find errors and glitches. Testers can set record demand as per schedule.

What are Black Box testing techniques?

There are various test case design techniques applied for black box testing:

a. Boundary Value Analysis
b. Equivalence partitioning
c. State Transition Testing
d. Decision Table Testing
e. Graph Based Testing
f. Error Guessing Technique

6.2.1 BVA OR BOUNDARY VALUES ANALYSIS

Taking inputs for a test case data for an age section should accept a valid data of anything between 1 100. According to BVP analysis, the software will be tested against four test data as 1, 1, 100, and 101 to check the system's response using the boundary values.

User id is =>100

Boundary Value Analysis		
Invalid	**Valid**	**Invalid**
1,0	1,2,9,100	11,12

6.2.2 EQUIVALENCE CLASS PARTITIONING

Taking inputs for a test case data for the example mentioned above will have three classes from which one data will be tested.

Valid class: 1 to 100 (any number), Invalid class: 1 (checking the lowest of lowest), Invalid class: 101(highest of highest).

Equivalence Class Partitioning		
Invalid	**Valid**	**Invalid**
1,0	1,2,9,100	11,12
13	33	12

6.2.3 STATE TRANSITION TESTING

A login page will let you input username and password until six attempts. Each incorrect password will be sent the user to the login page. After the seventh attempt, the user will be sent to an error page. This state transition method considers the various states of the system and the inputs to pass only the right sequence of the testing.

Tests	**Username**	**Password**
1	Wrong	Right
2	Right	Wrong
3	Wrong	Wrong
Access	Right	Right

6.2.4 DECISION TABLE TESTING

Decision table testing is a software testing technique used to test system behavior for different input combinations. This is a systematic approach where the different input combinations and their corresponding system behavior (Output) are captured in a tabular form. That is why it is also called as a Cause Effect table where Cause and effects are captured for better test coverage.

Conditions	**Rule 1**	**Rule 2**	**Rule 3**	**Rule 4**
Username	T	T	F	T
Password	F	F	T	T
Output	E	E	E	H

1. **T** – Correct username/password
2. **F** – Wrong username/password
3. **E** – Error message is displayed
4. **H** – Home screen is displayed

6.3 WHITE BOX TESTING

If we go by the definition, "White box testing" (also known as clear, glass box or structural testing) is a testing technique which evaluates the code and the internal structure of a program.

a. Statement Coverage
b. Branch Coverage
c. Path Coverage

The following table lists the advantages and disadvantages of white box testing.

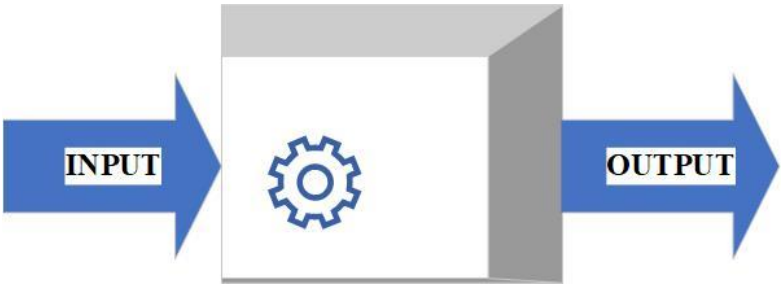


Figure: 6.3.1 WHITE BOX TESTING

ADVANTAGES

1. As the tester has knowledge of the source code, it becomes very easy to find out which type of data can help in testing the application effectively.
2. It helps in optimizing the code.
3. Extra lines of code can be removed which can bring in hidden defects.
4. Due to the tester's knowledge about the code, maximum coverage is attained during test scenario writing.

DISADVANTAGES

1. Due to the fact that a skilled tester is needed to perform white box testing, the costs are increased.
2. Sometimes it is impossible to look into every nook and corner to find out hidden errors that may create problems, as many paths will go untested.
3. It is difficult to maintain white box testing, as it requires specialized tools like code analyzers and debugging tools.

The application could be slow for the following reasons:

i. When logic is used.
ii. For the conditional cases, we will use or & and adequately.
iii. Switch case, which means we cannot use nested if, instead of using a switch case.

6.3.1 STATEMENT COVERAGE

Statement coverage is a white box testing technique that ensures all executable statements in the code are run and tested at least once. For example, if there are several conditions in a block of code, each of which is used for a certain range of inputs, the test should execute each and every range of inputs, to ensure all lines of code are actually executed.

6.3.2 BRANCH COVERAGE

Branch coverage is a testing method, which aims to ensure that each one of the possible branches from each decision point is executed at least once and thereby ensuring that all reachable code is executed. That is, every branch taken each way, true and false.

6.3.3 PATH COVERAGE

Path coverage is concerned with linearly independent paths through the code.As said earlier, path coverage is used to test the complex code snippets, which basically involve loop statements or combination of loops and decision statements.

CHAPTER 7

TOOLS AND TECHNOLOGIES

7.1 PROGRAMMING LANGUAGE

HTML, CSS, JavaScript, and PHP are foundational components in Visual Studio for developing a **Hospital Management System.** They play crucial roles in building web-based healthcare solutions, with HTML and CSS handling the structure and styling of the interface, JavaScript managing dynamic and interactive features, and PHP powering backend logic and database interactions.

7.1.1. Frontend in Hospital Management System Development (HTML, CSS, JavaScript):

HTML (HyperText Markup Language) serves as the backbone for creating the structure of all web pages in the hospital management system. It organizes elements such as patient registration forms, appointment schedules, billing pages, and staff dashboards. CSS (Cascading Style Sheets) is used to style these elements, ensuring a clean, responsive, and professional look across different devices. JavaScript adds interactivity and real-time functionality—such as live search for patient records, automatic appointment reminders, and dynamic content updates without page reloads.

7.1.2. Backend in Hospital Management System Development (PHP):

PHP (Hypertext Preprocessor) serves as the core programming language for backend development. It processes form submissions, manages database operations, handles authentication, and ensures secure data transactions. For example, when a doctor updates a patient's medical history, PHP scripts validate the input, update the database, and send confirmation to the frontend. PHP works seamlessly with databases like MySQL to store patient records, billing information, and appointment data securely.

7.1.3. Integration of Frontend and Backend:

The integration of HTML, CSS, JavaScript, and PHP within Visual Studio provides a robust and flexible framework for hospital management systems. The frontend communicates with the backend through HTTP requests, allowing real-time updates and data synchronization. For example, an appointment booking form created in HTML/CSS with interactive validation using JavaScript can instantly send data to a PHP script for processing and database storage. This division of responsibilities ensures maintainability, scalability, and secure handling of sensitive healthcare information.

7.1.4. Conclusion:

In conclusion, HTML, CSS, JavaScript, and PHP form a powerful technology stack for building a hospital management system in Visual Studio. HTML and CSS deliver a well-structured and visually appealing interface, JavaScript enables dynamic and responsive features, and PHP ensures reliable backend processing and secure database management. Together, these technologies create a comprehensive, user-friendly, and efficient platform that streamlines hospital operations, enhances communication between staff and patients, and improves overall healthcare service delivery.

7.2 OPERATING ENVIRONMENT:

Visual Studio Code (VS Code) is a lightweight yet powerful source code editor developed by Microsoft, widely used for developing web-based applications such as Hospital Management Systems (HMS). It supports numerous programming languages and frameworks, with rich features for coding, debugging, and integrating tools. Due to its flexibility, extensions, and speed, VS Code has become a popular choice for both individual developers and large teams when building secure and scalable healthcare solutions.

7.2.1. Components of Visual Studio Code for HMS Development

1) **Code Editor:**

   VS Code's code editor supports HTML, CSS, JavaScript, and PHP — the core languages for building a hospital management system. It provides syntax highlighting, intelligent code completion, refactoring, and real-time error detection, making coding more efficient and less error-prone.

2) **Extensions and Plugins:**

   Developers can install extensions such as PHP Intelephense, Live Server, Prettier, and MySQL tools to enhance productivity, improve code formatting, and manage databases directly from the editor.

3) **Integrated Terminal:**

   VS Code includes a built-in terminal for running PHP scripts, executing MySQL commands, and launching development servers without leaving the editor.

4) **Live Server Preview:**

   Using the Live Server extension, developers can instantly preview HMS web pages in the browser and see changes in real time.

5) **Version Control Integration:**

   VS Code integrates seamlessly with Git, enabling developers to manage code versions, track changes, collaborate with team members, and handle branching efficiently.

6) **Debugger:**

   Built-in debugging tools allow step-by-step execution of JavaScript code, as well as integration with PHP debugging extensions (e.g., Xdebug) for backend troubleshooting.

7) **Database Management Support:**

   With database extensions, developers can connect to MySQL or Apache databases to create, read, update, and delete hospital data directly from VS Code.

7.2.2. Operating Environment for HMS Development

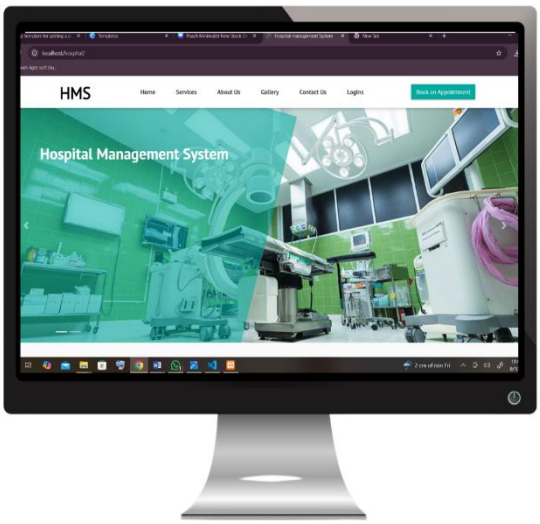
1) **Supported Operating Systems:**
2) **Windows:** Windows 8/10/11 (64-bit)
3) **MacOS:** MacOS 10.14 or higher
4) **Linux:** Popular distributions such as Ubuntu, Fedora, and Debian (64-bit)
5) **Software Requirements:**
6) **PHP:** Version 7.4 or higher (recommended 8.x)
7) **MySQL/Apache SQL:** For secure patient data and hospital records storage
8) **Web Server:** Apache or Nginx (often included in XAMPP or WAMP packages)
9) **Browser:** Latest versions of Chrome, Firefox, or Edge for testing the HMS frontend

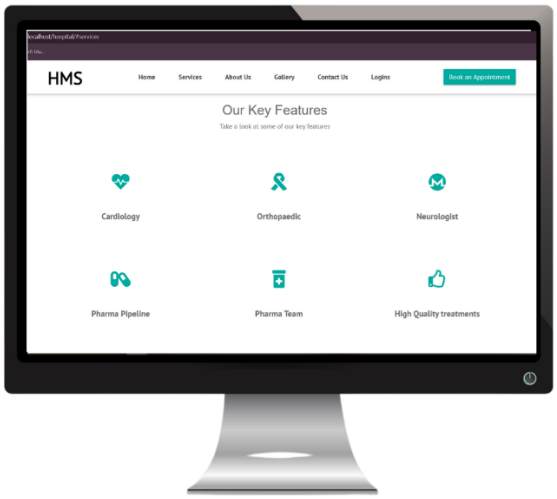
7.2.3.Recommended Specifications for Optimal Performance:

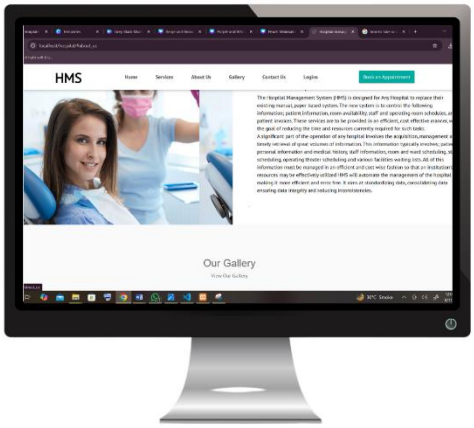
i. **CPU:** Intel Core i5 or AMD Ryzen 5 (or higher) for smooth multitasking
ii. **RAM:** 8 GB minimum, 16 GB recommended for handling multiple browser tabs, database tools, and VS Code extensions simultaneously
iii. **Storage:** Solid State Drive (SSD) with at least 10 GB of free space for project files, databases, and backups
iv. **Screen Resolution:** Full HD (1920×1080) or higher for comfortable workspace management

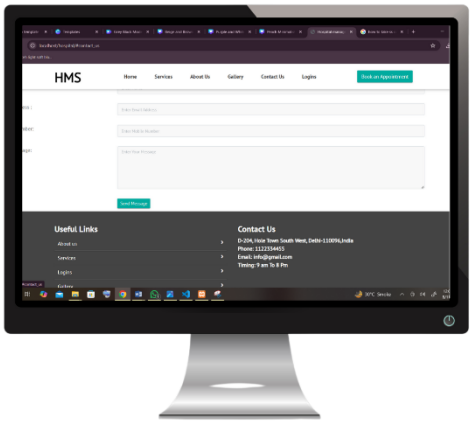
APPENDIX A

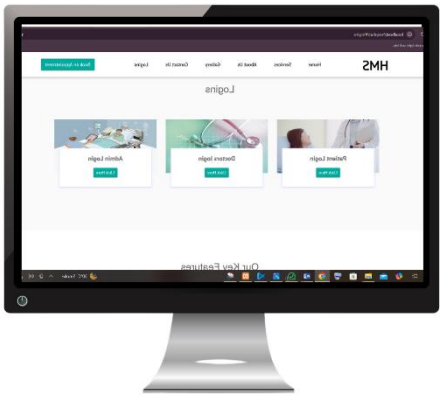
USER DOCUMENTATION

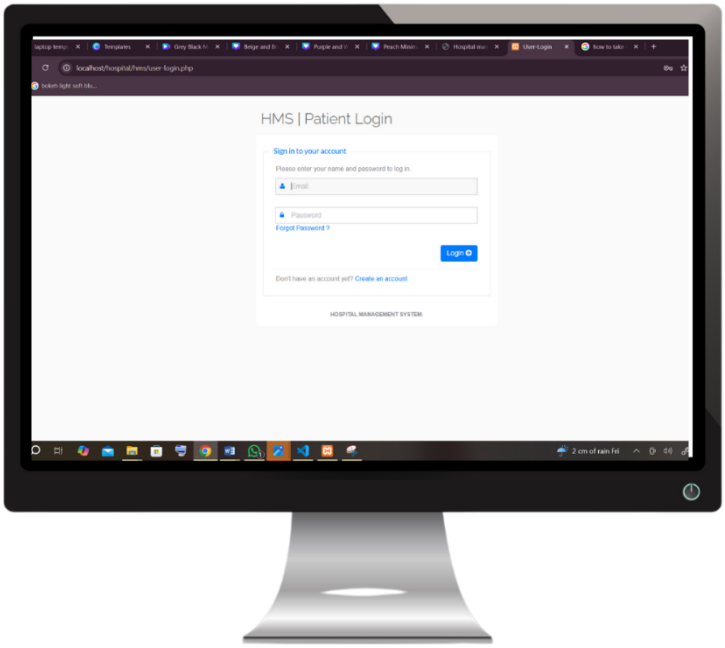

HMS
Home
Services
About Us
Gallery
Contact Us
Logins
Book an Appointment
Hospital Management System

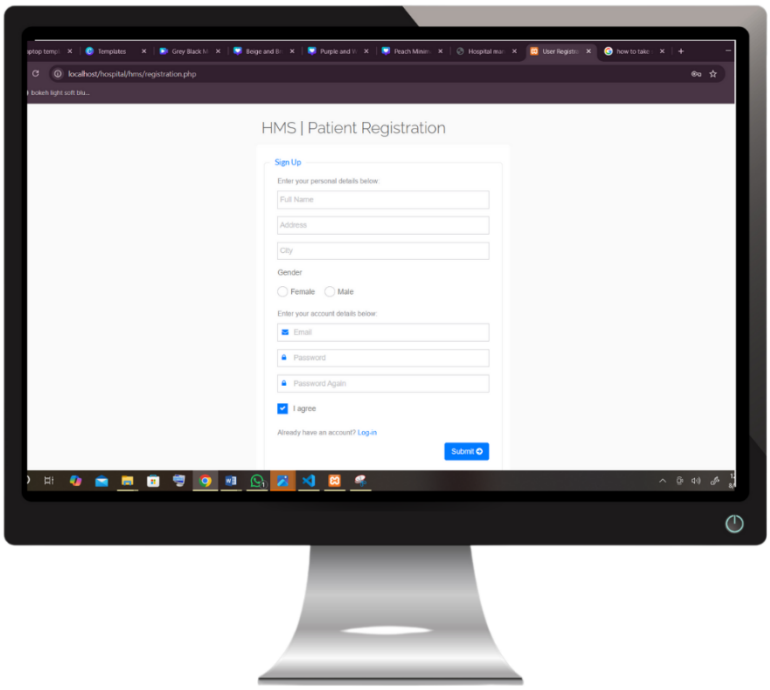

HMS
Home
Services
About Us
Gallery
Contact Us
Logins
Book an Appointment
Our Key Features
Take a look at some of our key features
Cardiology
Orthopaedic
Neurologist
Pharma Pipeline
Pharma Team
High Quality treatments

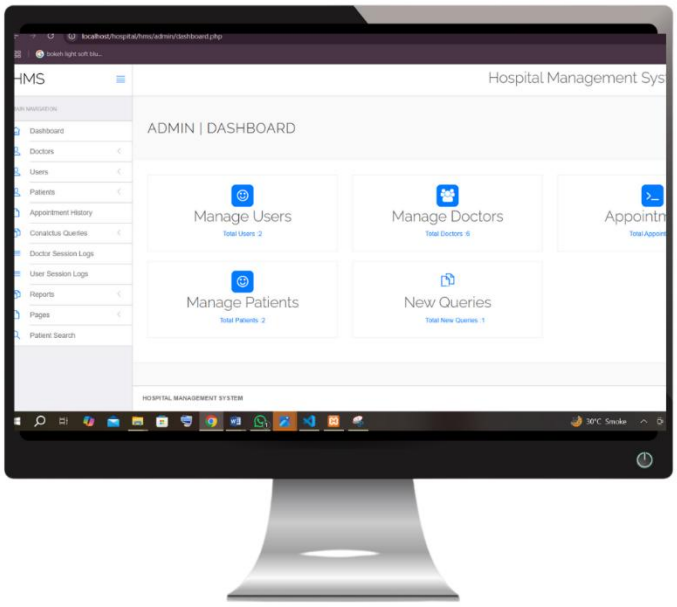

HMS
Our Gallery

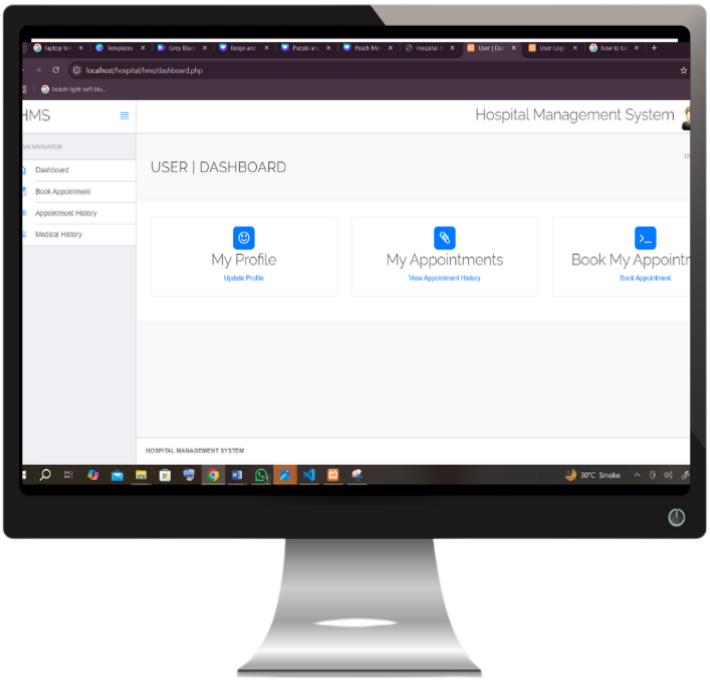

HMS
Useful Links
Contact Us




HMS | Patient Login
Sign in to your account
Login


HMS | Patient Registration
Sign Up
Enter your personal details below:
Full Name
Address
City
Gender
Female
Male
Enter your account details below:
Email
Password
Password Again
I agree
Already have an account? Log-in
Submit


HMS
Hospital Management Sys
ADMIN | DASHBOARD
Dashboard
Doctors
Users
Patients
Appointment History
Conactus Queries
Doctor Session Logs
User Session Logs
Reports
Pages
Patient Search
Manage Users
Total Users: 2
Manage Doctors
Total Doctors: 6
Manage Patients
Total Patients: 2
New Queries
Total New Queries: 1
HOSPITAL MANAGEMENT SYSTEM


HMS
Hospital Management System
Dashboard
Book Appointment
Appointment History
Medical History
USER | DASHBOARD
My Profile
Update Profile
My Appointments
View Appointment History
Book My Appointm
Book Appointment
HOSPITAL MANAGEMENT SYSTEM

APPENDIX B:

SOURCE CODE

HOME PAGE SOURCE CODE

```
<?php
include_once('hms/include/config.php');
if(isset($_POST['submit']))
{
$name=$_POST['fullname'];
$email=$_POST['emailid'];
$mobileno=$_POST['mobileno'];
$dscrption=$_POST['description'];
$query=mysqli_query($con,"insert into tblcontactus(fullname,email,contactno,message)
value('$name','$email','$mobileno','$dscrption')");
echo "<script>alert('Your information succesfully submitted');</script>";
echo "<script>window.location.href ='index.php'</script>";
} ?>
<!doctype html>
<html lang="en">
<head>
   <meta charset="utf-8">
   <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
   <title> Hospital management System </title>
   <link rel="shortcut icon" href="assets/images/fav.jpg">
   <link rel="stylesheet" href="assets/css/bootstrap.min.css">
   <link rel="stylesheet" href="assets/css/fontawsom-all.min.css">
    <link rel="stylesheet" href="assets/css/animate.css">
   <link rel="stylesheet" type="text/css" href="assets/css/style.css" />
</head>
   <body>
   <!-- ################## Header Starts Here#########################--->
```

```
<header id="menu-jk">
  <div id="nav-head" class="header-nav">
    <div class="container">
      <div class="row">
        <div class="col-lg-2 col-md-3 col-sm-12" style="color:#000;font-weight:bold;
font-size:42px; margin-top: 1% !important;">HMS
          <a data-toggle="collapse" data-target="#menu" href="#menu" ><i class="fas d-
block d-md-none small-menu fa-bars"></i></a>
        </div>
        <div id="menu" class="col-lg-8 col-md-9 d-none d-md-block nav-item">
          <ul>
            <li><a href="#">Home</a></li>
            <li><a href="#services">Services</a></li>
            <li><a href="#about_us">About Us</a></li>
            <li><a href="#gallery">Gallery</a></li>
            <li><a href="#contact_us">Contact Us</a></li>
            <li><a href="#logins">Logins</a></li>
          </ul>
        </div>
        <div class="col-sm-2 d-none d-lg-block appoint">
          <a class="btn btn-success" href="hms/user-login.php">Book an
Appointment</a>
        </div>
      </div>
    </div>
  </div>
</heade
 <!-- ################# Slider Starts Here#######################--->
<div class="slider-detail">
  <div id="carouselExampleIndicators" class="carousel slide" data-ride="carousel">
    <ol class="carousel-indicators">
      <li data-target="#carouselExampleIndicators" data-slide-to="0"
class="active"></li>
```

```
            <li data-target="#carouselExampleIndicators" data-slide-to="1"></li>
        </ol>
        <div class="carousel-inner">
            <div class="carousel-item ">
                <img class="d-block w-100" src="assets/images/slider/slider_2.jpg" alt="Second
slide">
                <div class="carousel-cover"></div>
                <div class="carousel-caption vdg-cur d-none d-md-block">
                    <h5 class="animated bounceInDown">Hospital Management System</h5>
                </div>
            </div>

            <div class="carousel-item active">
                <img  class="d-block  w-100"  src="assets/images/slider/slider_3.jpg"  alt="Third
slide">
                  <div class="carousel-cover"></div>
                <div class="carousel-caption vdg-cur d-none d-md-block">
                    <h5 class="animated bounceInDown">Hospital Management System</h5>
                </div>
            </div>
        </div>
        <a  class="carousel-control-prev"  href="#carouselExampleIndicators"  role="button"
data-slide="prev">
            <span class="carousel-control-prev-icon" aria-hidden="true"></span>
            <span class="sr-only">Previous</span>
        </a>
        <a  class="carousel-control-next"  href="#carouselExampleIndicators"  role="button"
data-slide="next">
            <span class="carousel-control-next-icon" aria-hidden="true"></span>
            <span class="sr-only">Next</span>
        </a>
      </div>
    </div>
```

```
<!--  ************************* Logins ************************** -->
  <section id="logins" class="our-blog container-fluid">
    <div class="container">
    <div class="inner-title">

        <h2>Logins</h2>
      </div>
      <div class="col-sm-12 blog-cont">
        <div class="row no-margin">
          <div class="col-sm-4 blog-smk">
            <div class="blog-single">
                <img src="assets/images/patient.jpg" alt="">
              <div class="blog-single-det">
                <h6>Patient Login</h6>
                <a href="hms/user-login.php" target="_blank">
                  <button class="btn btn-success btn-sm">Click Here</button>
                </a>
              </div>
            </div>
          </div>
          <div class="col-sm-4 blog-smk">
            <div class="blog-single">
                <img src="assets/images/doctor.jpg" alt="">
              <div class="blog-single-det">
                <h6>Doctors login</h6>
                <a href="hms/doctor" target="_blank">
                  <button class="btn btn-success btn-sm">Click Here</button>
                </a>
              </div>
            </div>
          </div>
          <div class="col-sm-4 blog-smk">
            <div class="blog-single">
```

```
                <img src="assets/images/admin.jpg" alt="">
            <div class="blog-single-det">
                <h6>Admin Login</h6>
                <a href="hms/admin" target="_blank">
                    <button class="btn btn-success btn-sm">Click Here</button>
                </a>
            </div>
        </div>
    </div>
  </div>
</section>
<!-- ################## Our Departments Starts Here#########################--->
<section id="services" class="key-features department">
  <div class="container">
    <div class="inner-title">

      <h2>Our Key Features</h2>
      <p>Take a look at some of our key features</p>
    </div>

    <div class="row">
      <div class="col-lg-4 col-md-6">
        <div class="single-key">
          <i class="fas fa-heartbeat"></i>
          <h5>Cardiology</h5>
        </div>
      </div>

     <div class="col-lg-4 col-md-6">
        <div class="single-key">
          <i class="fas fa-ribbon"></i>
```

```
                <h5>Orthopaedic</h5>
              </div>
            </div>
            <div class="col-lg-4 col-md-6">
              <div class="single-key">
                <i class="fab fa-monero"></i>
                <h5>Neurologist</h5>
              </div>
            </div>
            <div class="col-lg-4 col-md-6">
              <div class="single-key">
                <i class="fas fa-capsules"></i>
                <h5>Pharma Pipeline</h5>
              </div>
            </div>
            <div class="col-lg-4 col-md-6">
              <div class="single-key">
                <i class="fas fa-prescription-bottle-alt"></i>
                <h5>Pharma Team</h5>
              </div>
            </div>
            <div class="col-lg-4 col-md-6">
              <div class="single-key">
                <i class="far fa-thumbs-up"></i>
                <h5>High Quality treatments</h5>
              </div>
            </div>
          </div>
        </div>

    </section>
    <!--  ************************* About Us Starts Here *************************
-->
```

```
   <section id="about_us" class="about-us">
      <div class="row no-margin">
         <div class="col-sm-6 image-bg no-padding">
         </div>
         <div class="col-sm-6 abut-yoiu">
            <h3>About Our Hospital</h3>
<?php
$ret=mysqli_query($con,"select * from tblpage where PageType='aboutus' ");
while ($row=mysqli_fetch_array($ret)) {
?>
   <p><?php  echo $row['PageDescription'];?>.</p><?php } ?>
         </div>
      </div>
   </section>
         <!--************************Gallery-Starts-Here********************** -->
      <div id="gallery" class="gallery">
        <div class="container">
          <div class="inner-title">
            <h2>Our Gallery</h2>
            <p>View Our Gallery</p>
          </div>
          <div class="row">
      <div class="gallery-filter d-none d-sm-block">
         <button class="btn btn-default filter-button" data-filter="all">All</button>
         <button class="btn btn-default filter-button" data-filter="hdpe">Dental</button>
         <button class="btn btn-default filter-button" data-
filter="sprinkle">Cardiology</button>
         <button class="btn btn-default filter-button" data-filter="spray"> Neurology</button>
         <button class="btn btn-default filter-button" data-
filter="irrigation">Laboratry</button>
      </div>
      <br/>
         <div class="gallery_product col-lg-4 col-md-4 col-sm-4 col-xs-6 filter hdpe">
```

```
            <img src="assets/images/gallery/gallery_01.jpg" class="img-responsive">
        </div>
        <div class="gallery_product col-lg-4 col-md-4 col-sm-4 col-xs-6 filter sprinkle">
            <img src="assets/images/gallery/gallery_02.jpg" class="img-responsive">
        </div>
        <div class="gallery_product col-lg-4 col-md-4 col-sm-4 col-xs-6 filter hdpe">
            <img src="assets/images/gallery/gallery_03.jpg" class="img-responsive">
        </div>
        <div class="gallery_product col-lg-4 col-md-4 col-sm-4 col-xs-6 filter irrigation">
            <img src="assets/images/gallery/gallery_04.jpg" class="img-responsive">
        </div>
        <div class="gallery_product col-lg-4 col-md-4 col-sm-4 col-xs-6 filter spray">
            <img src="assets/images/gallery/gallery_05.jpg" class="img-responsive">
        </div>
        <div class="gallery_product col-lg-4 col-md-4 col-sm-4 col-xs-6 filter spray">
            <img src="assets/images/gallery/gallery_06.jpg" class="img-responsive">
        </div>
      </div>
    </div>
      </div>
      <!-- ####### Gallery End ####### -->
    <!--************************Contact-Us-Starts-Her***********************--!>

    <section id="contact_us" class="contact-us-single">
      <div class="row no-margin">

        <div  class="col-sm-12 cop-ck">
            <form method="post">
            <h2 >Contact Form</h2>
              <div class="row cf-ro">
                <div  class="col-sm-3"><label>Enter Name :</label></div>
                <div    class="col-sm-8"><input    type="text"    placeholder="Enter    Name"
name="fullname" class="form-control input-sm" required ></div>
```

```
                </div>
                <div  class="row cf-ro">
                    <div  class="col-sm-3"><label>Email Address :</label></div>
                    <div class="col-sm-8"><input type="text" name="emailid" placeholder="Enter
Email Address" class="form-control input-sm"  required></div>
                </div>
                 <div  class="row cf-ro">
                    <div  class="col-sm-3"><label>Mobile Number:</label></div>
                    <div          class="col-sm-8"><input          type="text"          name="mobileno"
placeholder="Enter Mobile Number" class="form-control input-sm" required ></div>
                </div>
                 <div  class="row cf-ro">
                    <div  class="col-sm-3"><label>Enter  Message:</label></div>
                    <div class="col-sm-8">
                      <textarea rows="5" placeholder="Enter Your Message" class="form-control
input-sm" name="description" required></textarea>
                    </div>
                </div>
                 <div  class="row cf-ro">
                    <div  class="col-sm-3"><label></label></div>
                    <div class="col-sm-8">
                     <button class="btn btn-success btn-sm" type="submit" name="submit">Send
Message</button>
                    </div>
                </div>
            </form>
            </div>
        </div>
    </section>
      <!-- ################# Footer Starts Here#######################--->
    <footer class="footer">
        <div class="container">
            <div class="row">
```

```
            <div class="col-md-6 col-sm-12">
               <h2>Useful Links</h2>
               <ul class="list-unstyled link-list">
                  <li><a  ui-sref="about"  href="#about">About  us</a><i  class="fa  fa-angle-
right"></i></li>
                  <li><a ui-sref="portfolio" href="#services">Services</a><i class="fa fa-angle-
right"></i></li>
                  <li><a  ui-sref="products"  href="#logins">Logins</a><i  class="fa  fa-angle-
right"></i></li>
                  <li><a  ui-sref="gallery"  href="#gallery">Gallery</a><i  class="fa  fa-angle-
right"></i></li>
                  <li><a ui-sref="contact" href="#contact">Contact us</a><i class="fa fa-angle-
right"></i></li>
               </ul>
            </div>
            <div class="col-md-6 col-sm-12 map-img">
               <h2>Contact Us</h2>
               <address class="md-margin-bottom-40">
<?php
$ret=mysqli_query($con,"select * from tblpage where PageType='contactus' ");
while ($row=mysqli_fetch_array($ret)) {
?>
                  <?php  echo $row['PageDescription'];?> <br>
                  Phone: <?php  echo $row['MobileNumber'];?> <br>
                  Email: <a href="mailto:<?php  echo $row['Email'];?>" class=""><?php  echo
$row['Email'];?></a><br>
                  Timing: <?php  echo $row['OpenningTime'];?>
               </address>
      <?php } ?>
            </div>
         </div>
      </div>
   </footer>
```

```
    <div class="copy">
            <div class="container">
        Hospital Management System
            </div>
        </div>
    </body>
<script src="assets/js/jquery-3.2.1.min.js"></script>
<script src="assets/js/popper.min.js"></script>
<script src="assets/js/bootstrap.min.js"></script>
<script src="assets/plugins/scroll-nav/js/jquery.easing.min.js"></script>
<script src="assets/plugins/scroll-nav/js/scrolling-nav.js"></script>
<script src="assets/plugins/scroll-fixed/jquery-scrolltofixed-min.js"></script>

<script src="assets/js/script.js"></script>

</html>
```